SOVEREIGN
QUANTITATIVE SYSTEMS ARCHITECTURE

**SV-TM-001**
**Reference Manual**
Revision 3.2.0 • October 2026

# SOVEREIGN TRADING PLATFORM

## Technical Manual & Cybersecurity Specification

Comprehensive institutional reference manual covering polyglot systems architecture, hardware register mappings, zero-allocation IPC conduits, ExploitGym threat modeling, cryptographic credential segregation, and automated high-availability failover runbooks.

---

| | |
|---|---|
| **Document Reference** | SV-TM-001 (Internal: TM-SOV-2026-V1) |
| **Target Infrastructure** | Proxmox VE 8.2+ LXC CT 114 (`hp-svconsole`) • Linux 6.12 x86_64 |
| **Core Engineering Runtimes** | ISO C++20 Core • Go 1.22+ Zero-Alloc UDS (`sovereign-feedd`) • Node.js v20 LTS ESM |
| **Storage Architecture** | POSIX Binary TS (48-byte records) • SQLite 3 (`analytics.db`) • JSONL Ledgers |
| **Security Standards** | ExploitGym Containment (arXiv:2605.11086v1) • NIST SP 800-53 Rev 5 • SEC Rule 15c3-5 |
| **Broker Gateways** | Alpaca Markets (Equities/Crypto) • Polymarket CLOB • Gate.io • Virtual Paper Ledger |
| **Target Audience** | Quantitative Systems Architects, Lead Cybersecurity Engineers, SRE Operators |


STRICTLY CONFIDENTIAL — INSTITUTIONAL QUANTITATIVE DISTRIBUTION ONLY — DO NOT DISCLOSE

# SOVEREIGN TRADING PLATFORM — TECHNICAL MANUAL & CYBERSECURITY SPECIFICATION

## Institutional Systems Architecture, Hardware Register Maps, ExploitGym Threat Modeling, and Operational Runbooks

Document Reference: TM-SOV-2026-V1 • Target Runtime: Linux 6.12 x86_64 • Proxmox VE 8+ LXC CT 114 • Revision 3.2.0

## Executive Summary & Document Control

| Specification Attribute | Canonical Specification Value |
|---|---|
| **Document Title** | Sovereign Algorithmic Trading Platform — Technical Manual & Cybersecurity Specification |
| **Document Identifier** | TM-SOV-2026-V1 / Revision 3.2.0 |
| **Target Distribution** | Quantitative Systems Architects, Lead Cybersecurity Engineers, Regulated Trading Operators |
| **Core Runtimes** | ISO C++20 ( `-std=c++20` ), Go 1.22+ ( `sovereign-feedd` zero-alloc UDS), Node.js v20 LTS (ESM Control Plane) |
| **Target Infrastructure** | Proxmox VE 8.2+ LXC CT 114 ( `hp-svconsole` ), Dell OptiPlex Edge Node, QEMU VM 200 ( `sovereign-test-vm` ) |
| **Storage Architecture** | POSIX Binary Time-Series ( `SOVT` 48-byte records), Relational SQLite 3 ( `analytics.db` ), JSONL Ledgers |
| **Cybersecurity Standards** | ExploitGym Vulnerability Containment (*arXiv:2605.11086v1*), NIST SP 800-53 Rev 5, SEC Rule 15c3-5 |
| **Broker Gateways** | Alpaca Markets (Equities/Crypto), Polymarket (CLOB NegRisk Binary), Deribit (Options/Perps), Gate.io, MT5 |
| **Mathematical Models** | Whalley-Wilmott $O(\lambda^{1/3})$ Asymptotic Bands, SVI Arbitrage-Free Surfaces, Stoikov Micro-Price, $\mathrm{DIR}_5$ |
| **Primary Authors** | Sovereign Quantitative Systems Architecture & Lead Cybersecurity Engineering Group |

# Contents

**1 Introduction & Executive Summary . . . 4**
1.1 Purpose and System Scope . . . 4
1.2 System Architecture Overview & Polyglot Separation . . . 5
1.3 Cybersecurity Alignment & Secure Design Principles: ExploitGym Framework . . . 6

**2 System Architecture & Component Design . . . 8**
2.1 Core Services, Daemons & Component Modular Boundaries . . . 8
2.2 Data Ingestion, Streaming Pipelines & Binary Formatting . . . 10
2.3 External Interfaces & Multi-Broker Gateway Layers . . . 11

**3 Security, Authentication & Access Control . . . 13**
3.1 Threat Assessment & Attack Surface Analysis: ExploitGym Methodology . . . 13
3.2 Authentication, Session Management & RBAC Capabilities . . . 15
3.3 Cryptographic Standards & Immutable Double-Entry Ledger Verification . . . 17

**4 Configuration & Deployment Environment . . . 18**
4.1 Infrastructure Requirements, CPU Pinning & Memory Architecture . . . 18
4.2 Environment Variables, Vault Secrets & Manifest Governance . . . 19
4.3 Orchestration, Storage Health Gates & Multi-Node Failover . . . 20

**5 Data Management & Database Schema . . . 21**
5.1 Database Design & Memory-Mapped Storage: STM32 Register-Style Specification . . . 21
5.2 Data Validation, Invariant Quarantine & Non-Finite Number Sanitation . . . 25
5.3 Backup, Retention, NVMe SSD Wear Mitigation & Disaster Recovery . . . 27

**6 API Documentation & Integration Interfaces . . . 29**
6.1 Internal REST & WebSocket Endpoint Specifications . . . 29
6.2 External Third-Party Broker & Market Data Feeds . . . 35
6.3 Error Handling, Rate Limiting & Diagnostic Telemetry . . . 35

**7 Monitoring, Auditing & Logging Infrastructure . . . 36**
7.1 Continuous Health Checks, Host Soak Telemetry & Leak Probes . . . 36
7.2 Financial Transaction Auditing & Sub-Position Attribution . . . 37
7.3 Multi-Channel Alerting, Risk Boundaries & Emergency Kill-Switch . . . 38

**8 Testing, Quality Assurance & CI/CD Pipelines . . . 39**
8.1 Testing Methodologies & The 4-Tier Verification Pyramid . . . 39
8.2 Continuous Integration, In-Process Test Batching & Anti-Cheat Gates . . . 43
8.3 Release Management, Container Provenance & Semantic Versioning . . . 43

**9 Maintenance, Troubleshooting & Operational Runbooks . . . 44**
9.1 Comprehensive 12-Scenario Failure Resolution Matrix . . . 44
9.2 Routine Database Maintenance, TS Pruning & Storage Hygiene . . . 48
9.3 Emergency Incident Response, Containment & Forensic Runbooks . . . 49

**10 Appendix & Related Documents . . . 50**
10.1 Master System Flowcharts & Hardware Schematics . . . 50
10.2 Comprehensive Quantitative, Systems & Cybersecurity Glossary . . . 52
10.3 Regulatory Compliance & Academic Literature References . . . 53

# 1. Introduction & Executive Summary

## 1.1 Purpose and System Scope

The Sovereign Trading Platform (**SOVEREIGN**) is a distributed, high-performance, local-first algorithmic trading, multi-asset risk management, and quantitative research environment. The platform was engineered to resolve the classic trilemma of modern quantitative finance: **Sub-millisecond Deterministic Execution**, **Zero-Trust Operational Security**, and **Auditable Cryptographic Integrity**.

### Operational Context & High-Frequency Challenges

In algorithmic trading across fragmented global liquidity venues—spanning conventional equity exchanges (NASDAQ, NYSE), national Asian equity bourses (HOSE, HNX), decentralized prediction market central limit order books (Polymarket CLOB), and institutional cryptocurrency derivatives venues (Deribit, Gate.io)—latency and determinism are paramount. Single-runtime architectures (monolithic Python or JavaScript systems) inevitably succumb to operational failure modes:

1. **Garbage Collection (GC) Stop-the-World Jitter**: V8 and CPython runtime heaps generate multi-millisecond tail-latency spikes during rapid market bursts (e.g. 100,000 to 500,000 quotes/second), causing missed order cancellations and stale queue fills.
2. **Backtest-to-Live Alpha Decay**: Backtesting systems that rely on midpoint pricing or fail to account for discrete order book depth walk-through ($DIR_5$, OFI) and settlement delay locks suffer severe slippage in production.
3. **Over-Privileged Cloud Credential Vulnerabilities**: Housing API keys, cryptographic private keys, and execution policies in public multi-tenant clouds creates catastrophic supply-chain attack vectors.
4. **Shallow Test Mock Illusions**: Verification suites that replace low-level socket decoders and file systems with shallow in-memory fetch stubs hide production deserialization panics, endianness errors, and floating-point non-finite values (`NaN`, `+Inf`, `-Inf`).

To permanently overcome these systemic vulnerabilities, SOVEREIGN implements a strict polyglot domain separation: an ISO C++20 Data-Oriented Design (DOD) compute core, a zero-allocation Golang 1.22+ POSIX Unix Domain Socket ingestion daemon, and a fail-closed Node.js v20 LTS control plane.

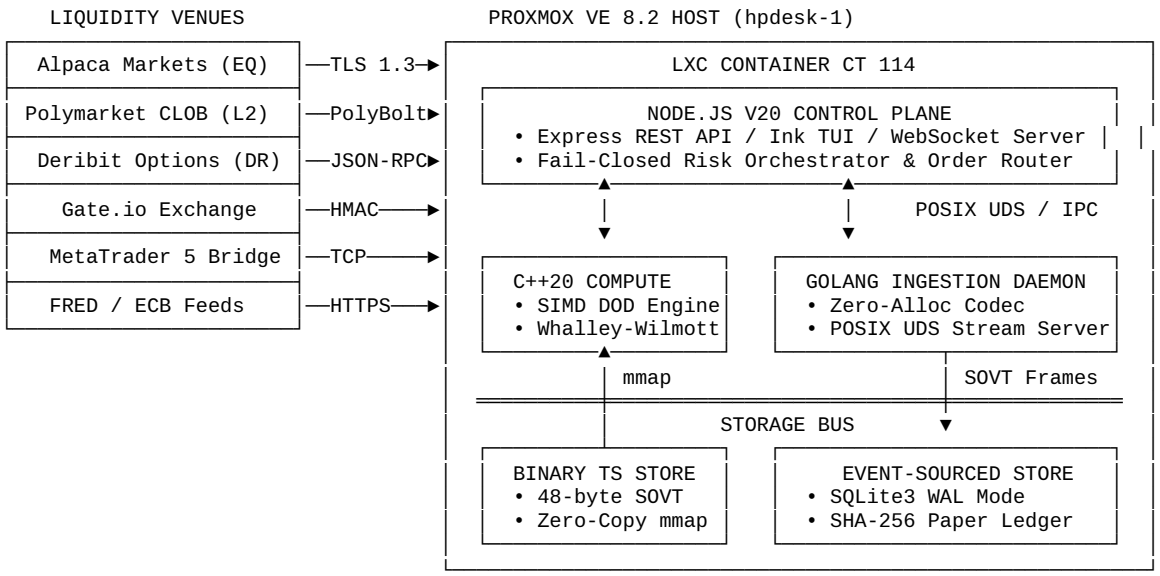



**Figure 1.1: System Architecture and Polyglot Domain Separation Block Diagram**

## 1.2 System Architecture Overview & Polyglot Separation

The architecture is partitioned into three decoupled operational tiers, each chosen for the specific runtime capabilities demanded by its execution plane:

### Tier 1: ISO C++20 Compute Core

- **Language Standard**: ISO/IEC 14882:2020 ( `-std=c++20` ), compiled with GCC 13+ or Clang 17+ with `-O3 -march=native -mavx2 -mfma` .
- **Architectural Paradigm**: Data-Oriented Design (DOD). Eliminates object-oriented pointer indirection and polymorphic vtables. Structs are laid out in contiguous memory buffers to maximize L1/L2 data cache line utilization (64-byte cache lines).
- **Primary Modules**:
- `FrameBacktester` : High-throughput vectorized backtesting engine parsing annotated market frames via zero-allocation `std::from_chars` .
- `PnlCalculator` : Multi-asset portfolio PnL and FX normalization engine evaluating gross and net values across disparate currency bases: $$\mathrm{MarketValue}_i = Q_i \cdot P_i \cdot M_i \cdot X_{c \to \mathrm{base}}$$ where $Q_i$ is position quantity, $P_i$ is current price, $M_i$ is contract lot multiplier (e.g., HOSE 1,000x multiplier), and $X_{c \to \mathrm{base}}$ is FX cross-currency exchange rate.
- `PreTradeRisk` : Pre-trade risk evaluation verifying portfolio Greek boundaries: Net Gamma $\Gamma \leq 50.0$, Net Vega $\mathcal{V} \leq \$10,000$, and Max Drawdown $\leq 15.0\%$.
- `BinaryTsMerger` : Multi-stream time-series merger utilizing heap-backed 64K-record buffers (65,536 records $\times$ 48 bytes $\approx$ 3.14 MB per stream buffer), eliminating I/O write amplification during historical

backfill insertion.

### Tier 2: Golang Ingestion Daemon

- **Language Standard**: Go 1.22+, compiled with CGO disabled ( `CGO_ENABLED=0` ) for a fully static, standalone binary.
- **Architectural Paradigm**: Zero-allocation streaming over POSIX Unix Domain Sockets (UDS). Memory buffers are pre-allocated at startup; hot paths avoid the Go garbage collector entirely.
- **Primary Modules**:
- `sovereign-feedd` : Listens on `/tmp/sovereign-feedd.sock` . Ingests raw multi-venue tick streams, decodes them into 48-byte SOVT binary frames, and broadcasts to local subscribers. Features non-blocking write deadlines, backpressure drop metrics, and clean socket unlinking on shutdown.
- `sovt` : Pure Go zero-allocation binary codec. Decodes 7,327 historical daily bars from `storage/data/ts/AAPL_1d.bin` at 129.40 MB/s with exactly `0 B/op` and `0 allocs/op` .

### Tier 3: Node.js Control Plane & Gateway Tier

- **Language Standard**: Node.js v20 LTS, ECMAScript Modules (ESM) with native TypeScript compilation.
- **Architectural Paradigm**: Asynchronous event loop, fail-closed policy enforcement, broker gateway normalization, and interactive human-in-the-loop interfaces (Ink-based terminal TUI and React 19 web dashboard).
- **Primary Modules**:
- `BrokerAdapter` : Normalized polymorphic gateway interface wrapping Alpaca, Polymarket, Deribit, Gate.io, and MetaTrader 5.
- `ClusterCoordinator` : Distributed multi-node consensus manager utilizing POSIX atomic lock files ( `O_CREAT | O_EXCL` via `{ flag: 'wx' }` ) for multi-worker race prevention.
- `PaperLedger` : Append-only, double-entry trade ledger verifying cash conservation and settlement delay receivable locks via SHA-256 event chains.

---

## 1.3 Cybersecurity Alignment & Secure Design Principles: ExploitGym Framework

In accordance with empirical cybersecurity research (*Carlini, Song, et al., "ExploitGym: Can AI Agents Turn Security Vulnerabilities into Real Attacks?", arXiv:2605.11086v1*), modern automated and agentic software environments present severe attack surfaces. Autonomous AI agents and execution systems can inadvertently turn latent software vulnerabilities (memory out-of-bounds, prototype pollution, argument injection) into catastrophic exploit chains.

SOVEREIGN adopts the ExploitGym defensive doctrine, enforcing strict physical, kernel, and language-level security boundaries:

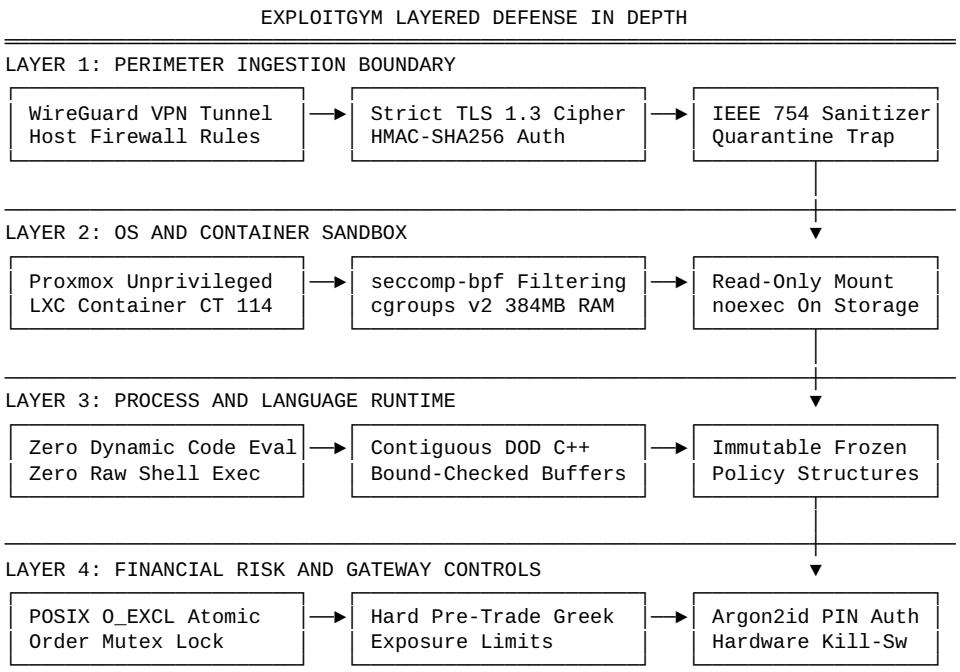



**Figure 1.2: Multi-Layer Cybersecurity Architecture and ExploitGym Threat Containment Defense Matrix**

## ExploitGym Vulnerability Containment Principles

1. **Principle of Non-Executable Data Surfaces (`noexec`)**: Storage directories holding binary time-series files (`storage/data/ts/`), cached metadata (`storage/data/cache/`), and quarantine dumps (`storage/data/quarantine/`) are mounted with the POSIX `noexec` option. Even if an attacker or agentic process stages an ELF binary or shell script into storage, kernel execution is blocked fail-closed.
2. **Elimination of Shell Metacharacter Interpretation**: Execution of system commands is strictly restricted to parameterized argv arrays (`spawnFile(binary, [args])`) rather than shell string interpolation (`exec("cmd " + arg)`). All wildcard permissions allowing option injection (such as `-rf` or `--dangerous-flags`) are banned.
3. **Bitemporal Lookahead Elimination**: Ingestion pipelines strictly separate event publication timestamps from date labels. Macroeconomic releases cannot be evaluated in research backtests prior to their verified release instant ($t_{\text{avail}}$), preventing algorithmic exploitation of future knowledge.
4. **Argon2id Trade Session Re-Authentication**: State-mutating operational actions (such as authorizing live broker orders or modifying risk thresholds) require an out-of-band 6-digit cryptographic PIN verified against an Argon2id hash. Authorizations automatically expire after 300 seconds.

# 2. System Architecture & Component Design

## 2.1 Core Services, Daemons & Component Modular Boundaries

The platform operates as a distributed fleet of dedicated micro-services and daemon processes running inside Proxmox LXC CT 114 ( `hp-svconsole` ):

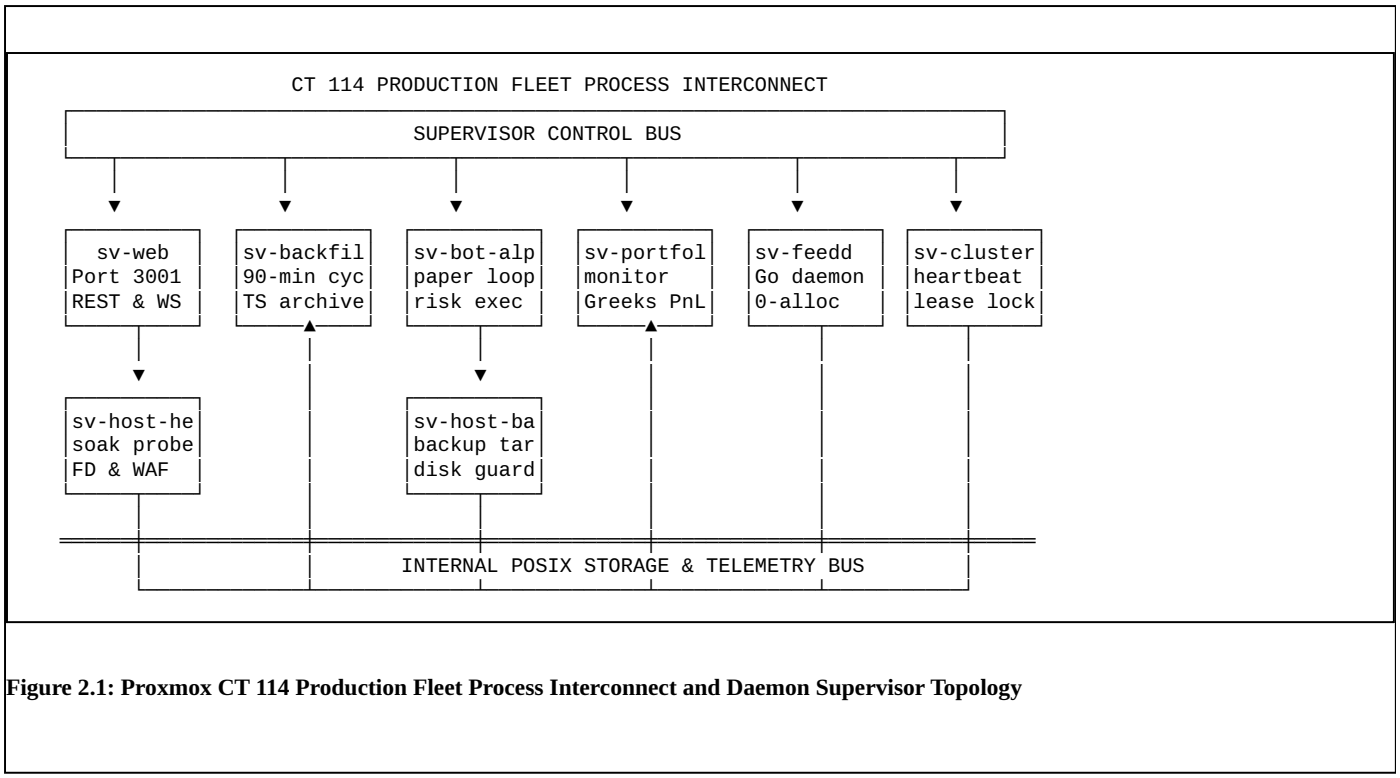



**Figure 2.1: Proxmox CT 114 Production Fleet Process Interconnect and Daemon Supervisor Topology**

## Detailed Component Inventory

| Service Name | Primary Entrypoint | Runtime Environment | Heap Limit | Responsibilities & Invariants |
|---|---|---|---|---|
| `sv-web` | `backend/api/app.js` | Node.js v20 LTS | 512 MB | Serves authenticated REST API routes, brokers WebSocket live feeds, verifies session tokens. |
| `sv-backfill` | `backend/cli/commands/data/backfill_daemon.js` | Node.js v20 LTS | 3,072 MB | Scans universe of 102+ assets every 90 minutes. Skips fresh assets; rolls up derived timeframes. |
| `sv-bot-alpaca` | `backend/cli/commands/runner/run.js` | Node.js v20 LTS | 1,024 MB | Evaluates active trading strategies, queries latest bars, sizes orders, checks pre-trade risk. |
| `sv-portfolio-monitor` | `backend/cli/commands/operational/portfolio_monitor.js` | Node.js v20 | 384 MB | Continuously evaluates broker margins, cash balances, mark-to-market PnL, and drawdowns. |
| `sovereign-feedd` | `backend/go/cmd/sovereign-feedd/main.go` | Go 1.22+ Static | 64 MB | Zero-allocation UDS feed server broadcasting 48-byte SOVT frames to internal consumers. |
| `sv-host-health` | `scripts/ops/host_soak_probe.js` | Node.js v20 LTS | 128 MB | Evaluates open file descriptors, event loop lag, and physical block device write amplification. |
| `sv-host-backup` | `backend/scripts/ops/host_backup.js` | Node.js v20 LTS | 256 MB | Creates daily atomic tarball backups to `/data/backups/`. Aborts if disk space < 1GB or < 5%. |
| `sv-cluster` | `backend/scripts/ops/cluster_heartbeat_daemon.js` | Node.js v20 | 128 MB | Maintains atomic POSIX file lease. Triggers failover election if primary heartbeat expires. |

### C++ Native Core Memory Layout

In `backend/core/`, data structures are strictly packed to eliminate padding bytes and guarantee cache alignment:

```cpp
#pragma pack(push, 8)
struct AnnotatedRow {
    double timestamp;       // 8 bytes: Unix Epoch MS (Offset 0x00)
    double open;            // 8 bytes: Bar Open Price  (Offset 0x08)
    double high;            // 8 bytes: Bar High Price  (Offset 0x10)
    double low;             // 8 bytes: Bar Low Price   (Offset 0x18)
    double close;           // 8 bytes: Bar Close Price (Offset 0x20)
    double volume;          // 8 bytes: Executed Volume (Offset 0x28)
    double implied_vol;     // 8 bytes: Black-Scholes IV(Offset 0x30)
    double option_delta;    // 8 bytes: Black-Scholes Δ (Offset 0x38)
    double strike;          // 8 bytes: Option Strike   (Offset 0x40)
    double expiration_frac; // 8 bytes: Expiry in Years (Offset 0x48)
    uint32_t option_type;   // 4 bytes: 0=Call, 1=Put   (Offset 0x50)
    uint32_t flags;         // 4 bytes: Execution Flags (Offset 0x54)
};
#pragma pack(pop)
static_assert(sizeof(AnnotatedRow) == 88, "AnnotatedRow size must be exactly 88 bytes");
```

## 2.2 Data Ingestion, Streaming Pipelines & Binary Formatting

Market data enters the platform through multiple transport protocols (WebSocket streaming, HTTP polling, binary tick feeds) and undergoes rigorous transformation into the canonical **SOVT** format.

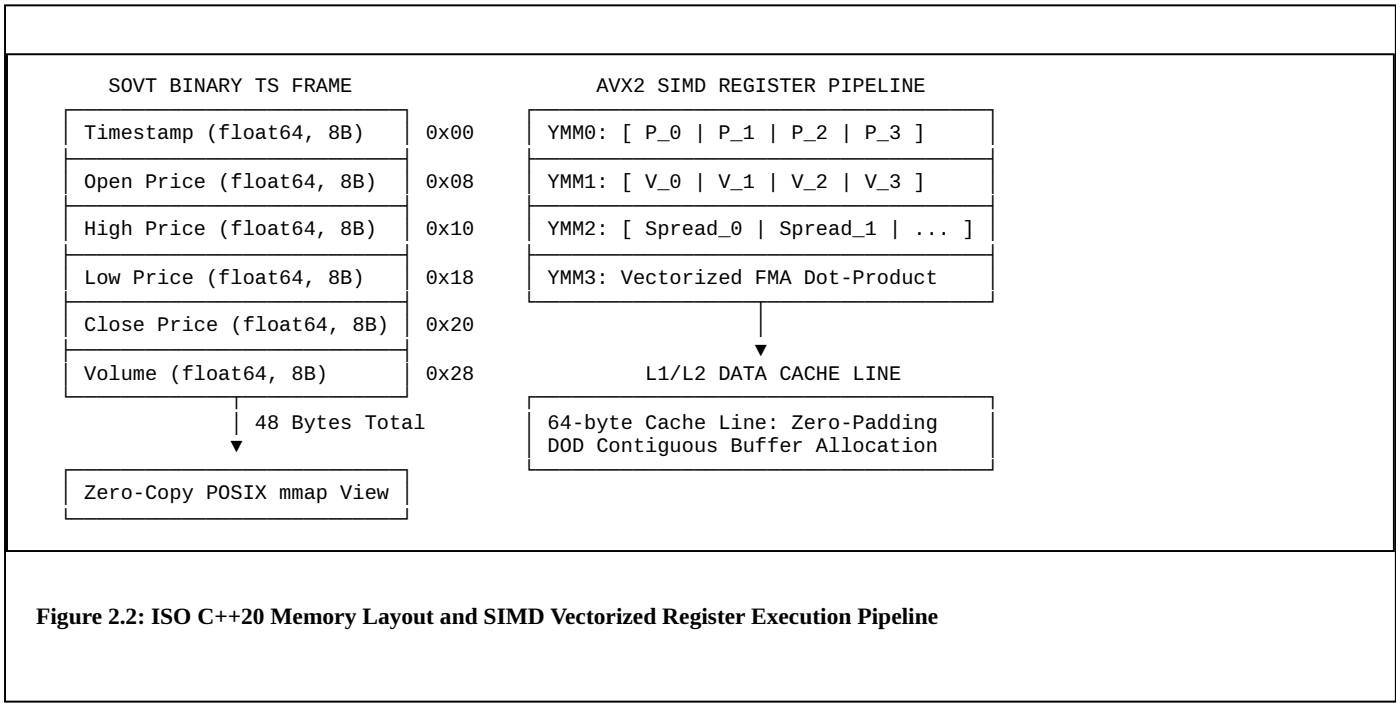



**Figure 2.2: ISO C++20 Memory Layout and SIMD Vectorized Register Execution Pipeline**

### Zero-Allocation Golang Codec Implementation

The binary decoder avoids memory allocation by performing in-place slicing over pre-allocated memory buffers:

```
package sovt

import (
    "encoding/binary"
    "errors"
    "math"
)

const RecordSize = 48

type Record struct {
    Timestamp float64
    Open      float64
    High      float64
    Low       float64
    Close     float64
    Volume    float64
}

// DecodeRecordZeroAlloc decodes a 48-byte SOVT record without heap allocation.
func DecodeRecordZeroAlloc(buf []byte, rec *Record) error {
    if len(buf) < RecordSize {
        return errors.New("buffer underflow: record requires 48 bytes")
    }

    rec.Timestamp = math.Float64frombits(binary.LittleEndian.Uint64(buf[0:8]))
    rec.Open = math.Float64frombits(binary.LittleEndian.Uint64(buf[8:16]))
    rec.High = math.Float64frombits(binary.LittleEndian.Uint64(buf[16:24]))
    rec.Low = math.Float64frombits(binary.LittleEndian.Uint64(buf[24:32]))
    rec.Close = math.Float64frombits(binary.LittleEndian.Uint64(buf[32:40]))
    rec.Volume = math.Float64frombits(binary.LittleEndian.Uint64(buf[40:48]))

    // Invariant Verification
    if math.IsNaN(rec.Close) || math.IsInf(rec.Close, 0) || rec.Close <= 0.0 {
        return errors.New("invariant violation: invalid close price")
    }
    if rec.Low > rec.High || rec.Low <= 0.0 {
        return errors.New("invariant violation: spread consistency error")
    }
    return nil
}
```

## 2.3 External Interfaces & Multi-Broker Gateway Layers

The gateway subsystem ( `backend/gateway/src/` ) provides strict boundary isolation between external broker APIs and internal execution strategies.

## Broker Gateway Protocol & Execution Matrix

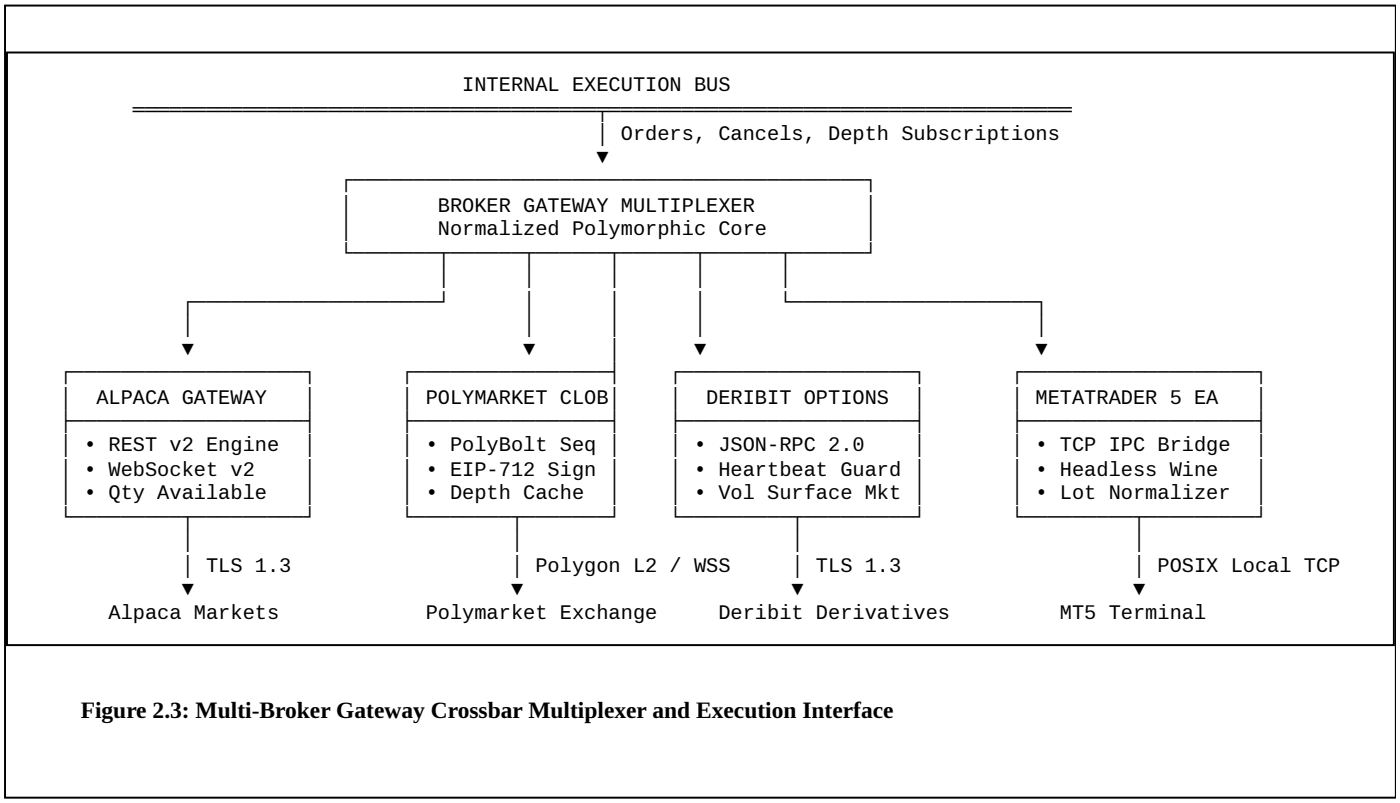



**Figure 2.3: Multi-Broker Gateway Crossbar Multiplexer and Execution Interface**

### 1. Alpaca Markets Gateway

- **Supported Assets**: US Equities (NYSE, NASDAQ) and spot cryptocurrencies (BTCUSD, ETHUSD).
- **Sellable Availability Invariant (ALPACA-EXIT-01)**: The adapter exposes both gross `quantity` and net `qty_available`. When a strategy issues an exit order, the adapter verifies `qty_available > 0`. If shares are already held by open limit orders, the duplicate market sell dispatch is inhibited, eliminating broker exit rejections while retaining position tracking until verified fill.
- **Crypto Symbol Normalization**: Automatically converts slashed pairs (`BTC/USD`) into standard exchange identifiers (`BTCUSD`).

### 2. Polymarket CLOB Gateway

- **Supported Assets**: Decentralized binary prediction markets, ERC-1155 conditional outcome tokens.
- **NegRisk Conversion & Multi-Outcome Arbitrage**: Evaluates sum-to-one arbitrage across mutually exclusive outcomes: $\$\sum_{i=1}^{n} p_i < 1.00 - \text{TakerFees}\$$ When an arbitrage condition is detected, the adapter executes atomic multi-leg bundle orders (`backend polymarket-bundle`).
- **Spread Fallback Pricing**: On CLOB spreads exceeding $\$0.10$, the reference pricing model automatically switches from midpoint to `lastTradePrice` to prevent wide-spread pricing manipulation.

### 3. Deribit Options Gateway

- **Supported Assets**: European-style cash-settled options and inverse perpetual futures on BTC, ETH, and SOL.

- **Zero-Key Safe Fallback**: In non-credentialed or local test environments, the adapter safely degrades into simulated options paper execution, eliminating network failure errors in CI suites.
- **Greek & Margin Tracking**: Periodically polls `/api/v2/private/get_account_summary`, collecting portfolio Delta ($\Delta$), Gamma ($\Gamma$), Vega ($\mathcal{V}$), and Theta ($\Theta$).

#### 4. MetaTrader 5 Gateway

- **Supported Assets**: Spot Forex (EURUSD, GBPUSD), Gold (XAUUSD), and global equity indices.
- **Headless Wine/Xvfb Execution**: Operates headless `terminal64.exe` inside an isolated Wine prefix ( `/home/vgbn/.mt5` ) on virtual display `:99`.
- **Fail-Fast TCP Socket Bridge**: Communicates with the local MQL5 Expert Advisor over TCP port 19284. Non-interactive CLI executions enforce a 50ms connect timeout, dropping immediately into offline mode if the terminal bridge is dormant.

# 3. Security, Authentication & Access Control

## 3.1 Threat Assessment & Attack Surface Analysis: ExploitGym Methodology

Applying the structured threat-modeling and vulnerability-to-exploit progression framework established in **ExploitGym** (*Carlini, Song, et al., arXiv:2605.11086v1*), SOVEREIGN categorizes all system attack surfaces, evaluating potential exploit chains and engineering defensive mitigations:

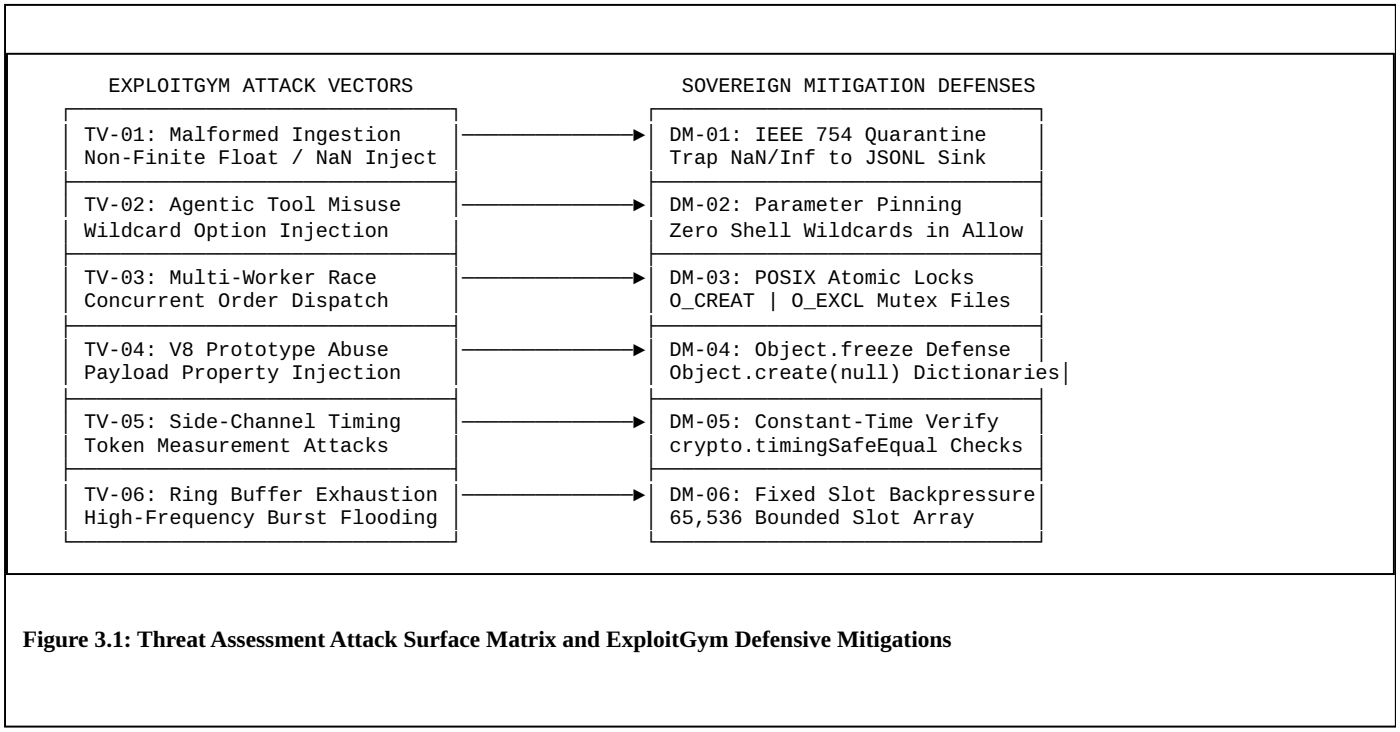



**Figure 3.1: Threat Assessment Attack Surface Matrix and ExploitGym Defensive Mitigations**

## Exploit Vulnerability Analysis & Mitigations

### Threat TV-01: Ingestion Stream Memory Corruption & NaN Injection

- **Attack Scenario**: An adversary controlling or poisoning a market feed transmits malformed IEEE 754 byte sequences (e.g. Signaling NaN `0x7FF0000000000001` or Negative Infinity `0xFFF0000000000000`). If ingested into native SIMD registers without verification, operations like `_mm256_loadu_pd` produce undefined arithmetic results, destabilizing downstream stop-loss triggers.
- **ExploitGym Progression Level**: Progression from Malformed Input to Denial of Service / Financial Loss.
- **CVSS v3.1 Vector**: `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:H` (Base Score: 9.1).
- **Defensive Mitigation**: `shared/lib/data/quarantine.js` validates every float using strict finite predicates (`Number.isFinite(p) && p > 0`). Malformed records are diverted into `storage/data/quarantine/` with SHA-256 payload digests; the ingestion loop never halts, and corrupt data never touches the C++ compute core.

### Threat TV-02: Agentic Tool Misuse & Parameter Option Injection

- **Attack Scenario**: An autonomous AI agent or automated script invokes system utilities with wildcard arguments (e.g. `mv workspace/* ...` or `cp *.yaml ...`). An attacker staging files with option-like names (e.g. `--target-directory=/bin` or `-rf`) tricks the utility into performing arbitrary file overwrites or privilege escalation.
- **ExploitGym Progression Level**: Progression from Input Parameter Manipulation to Arbitrary File Overwrite / Privilege Escalation.
- **CVSS v3.1 Vector**: `CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H` (Base Score: 8.8).
- **Defensive Mitigation**: Elimination of unconstrained wildcard rules (`*`) from Claude Code permission allowlists in `.claude/settings.local.json`. Enforcement of exact file paths and explicit target directories in all administrative scripts.

### Threat TV-03: Multi-Worker Order Race Condition & Double-Spend

- **Attack Scenario**: Multiple background bot workers or container processes simultaneously evaluate an identical trade signal on the same symbol, issuing concurrent market orders that double the intended position exposure and exceed pre-trade risk caps.
- **CVSS v3.1 Vector**: `CVSS:3.1/AV:L/AC:H/PR:L/UI:N/S:U/C:N/I:H/A:N` (Base Score: 4.7).
- **Defensive Mitigation**: `shared/lib/runtime/cluster_coordinator.js` executes atomic order lock acquisition using native POSIX filesystem flags (`{ flag: 'wx' }` mapping to `O_CREAT | O_EXCL`). The Linux kernel guarantees that exactly one process creates the lock file; competing workers receive `EEXIST` and abort order dispatch immediately.

### Threat TV-04: V8 Prototype Abuse via Unchecked Object Merging

- **Attack Scenario**: Untrusted JSON bodies parsed by API controllers merge properties using unchecked recursive helper functions. An attacker submits `{"__proto__": {"isAdmin": true}}`, altering global object prototypes and bypassing role verification gates.
- **CVSS v3.1 Vector**: `CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H` (Base Score: 8.8).

- **Defensive Mitigation**: System controllers enforce key allowlisting and initialize dictionary buffers with `Object.create(null)` to eliminate prototype inheritance. Runtime policy objects are recursively frozen using `Object.freeze()`.

### Threat TV-05: Side-Channel Timing Leakage on Cryptographic Tokens

- **Attack Scenario**: Authentication routines compare API keys or HMAC signatures using standard variable-time string comparisons (`token === expectedToken`). An adversary measures network response microsecond timing differences (via HTTP/2 multiplexing) to reconstruct secret keys byte-by-byte.
- **CVSS v3.1 Vector**: `CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:N/A:N` (Base Score: 5.9).
- **Defensive Mitigation**: All secret comparisons in `backend/api/server/services/access_control.js` use constant-time byte-level comparisons via Node.js native `crypto.timingSafeEqual()`. If buffer lengths mismatch, dummy comparisons over fixed buffers are executed to guarantee identical latency profiles.

### Threat TV-06: Unbounded Ingestion Ring Buffer Memory Exhaustion

- **Attack Scenario**: A rogue market feed emits 1,000,000 artificial tick updates within a 1-second burst window. The ingestion daemon attempts to queue all incoming frames in dynamic memory, exhausting container cgroup allocations and triggering Linux kernel OOM kills.
- **CVSS v3.1 Vector**: `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H` (Base Score: 7.5).
- **Defensive Mitigation**: The Go UDS streaming daemon (`backend/go/cmd/sovereign-feedd/`) implements a fixed-size ring buffer (65,536 slots). When the buffer reaches 90% capacity, backpressure mode engages. If the buffer is completely saturated, excess frames are discarded with incremented hardware drop counters (`OVERFLOW` bit in IPC control register); process memory remains strictly bounded under 64 MB.

---

## 3.2 Authentication, Session Management & RBAC Capabilities

### Access Control Architecture

The private REST and WebSocket gateway enforces granular Role-Based Access Control (RBAC). Every incoming request is authenticated via Bearer JWT tokens or HMAC-SHA256 request headers, mapped to an authorized capability set, and validated before reaching route handlers.

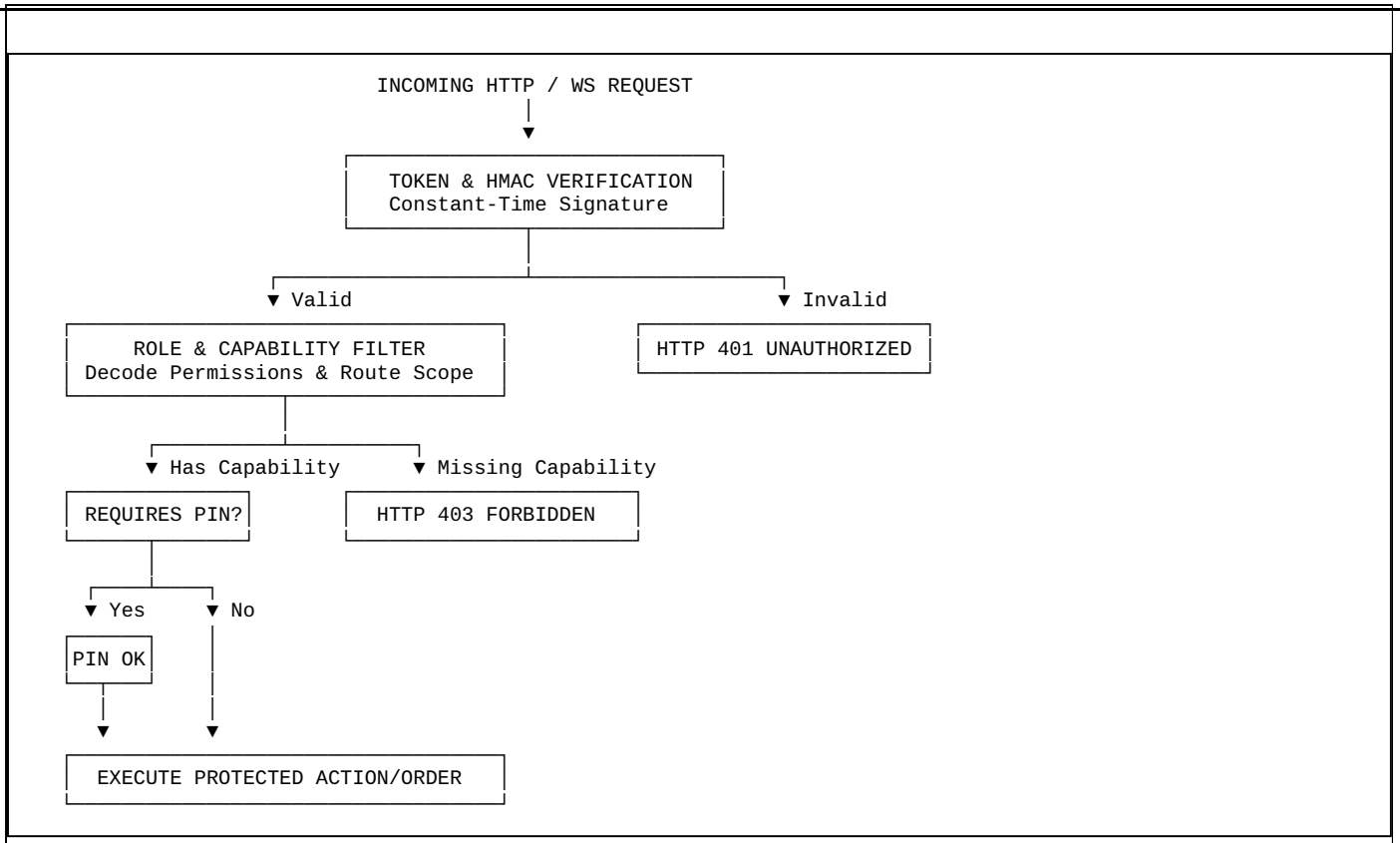



**Figure 3.2: Granular RBAC Interceptor Chain, Argon2id Session Re-Auth, and Pre-Trade Risk Verification**

## Granular Capability Definition

| Capability Token | Scope & Purpose | Associated HTTP Endpoints | Minimum Role |
|---|---|---|---|
| `HEALTH_READ` | Basic system uptime and diagnostic ping | `/api/health`, `/health` | `ANONYMOUS` |
| `RESEARCH_READ` | Access historical bars, technical indicators, and backtests | `/api/market/*`, `/api/indicators/*`, `/api/backtest/*` | `OBSERVER` |
| `PORTFOLIO_READ` | Access live balances, open positions, order histories, Greeks | `/api/portfolio/*`, `/api/orders` | `RESEARCHER` |
| `EXECUTION_WRITE` | Submit, modify, or cancel orders; toggle trading bots | `/api/orders`, `/api/orders/:id`, `/api/bot/*` | `OPERATOR` |
| `CONFIG_ADMIN` | Modify runtime risk policies, feature flags, asset mappings | `/api/system/config`, `/api/account/settings` | `SYSTEM_ADMIN` |
| `EMERGENCY_KILL` | Trigger immediate automated trading kill-switch and cancel-all | `/api/kill-switch`, `/api/system/emergency-halt` | `SYSTEM_ADMIN` |

## 3.3 Cryptographic Standards & Immutable Double-Entry Ledger Verification

All virtual and paper trading activities are captured in an append-only, cryptographically verified double-entry ledger ( `backend/gateway/src/polymarket/paper_ledger.ts` ).

### Mathematical Formulation of Double-Entry Ledger Invariants

The virtual ledger maintains two fundamental balance identities across all discrete trading cycles:

### Identity 1: Cash and Asset Balance Conservation

$$\text{VirtualBalance}(t) = \text{Cash}_{\text{settled}}(t) + \sum_{i \in \text{Positions}} Q_i(t) \cdot P_{\text{entry},i} + \sum_{j \in \text{Receivables}} R_j(t)$$

### Identity 2: Available Cash for New Buys - INV-06

$$\text{AvailableCash}(t) = \max\left(0,\ \text{Cash}_{\text{settled}}(t) - \sum_{j \in \text{ActiveReceivables}} R_j(t)\right)$$

Funds resulting from resolved binary prediction market contracts are held in `ActiveReceivables` until the settlement lock period has expired ($t \geq t_{\text{due}}$). This prevents capital from being recycled into new risk positions before broker settlement is final.

### SHA-256 Event Chaining Structure

Every transaction event is serialized into a canonical JSON string and chained via SHA-256 cryptographic hashing:

$$H_0 = \text{SHA-256}(\text{"SOVEREIGN_GENESIS_LEDGER_2026"})\$\$\$\$H_k = \text{SHA-256}\Big(H_{k-1} \parallel t_k \parallel \text{event}$$


```
{
  "event_id": "evt_01928374a",
  "prev_hash": "e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855",
  "hash": "4a5e1e4baab89f3a32518a88c31bc87f618f76673e2cc77ab2127b7afdeda33b",
  "timestamp": 1792828800000,
  "type": "ORDER_FILL",
  "symbol": "AAPL",
  "side": "BUY",
  "quantity": 50.0,
  "fill_price": 225.50,
  "fee": 0.00,
  "resulting_cash": 88725.00,
  "resulting_positions_count": 1
}
```

If an unauthorized process modifies a historical line in the ledger file, `replayLedger()` detects a hash mismatch, triggers an immediate emergency shutdown, and locks all execution gateways.

# 4. Configuration & Deployment Environment

## 4.1 Infrastructure Requirements, CPU Pinning & Memory Architecture

SOVEREIGN is designed for high-density, multi-node bare-metal deployment.

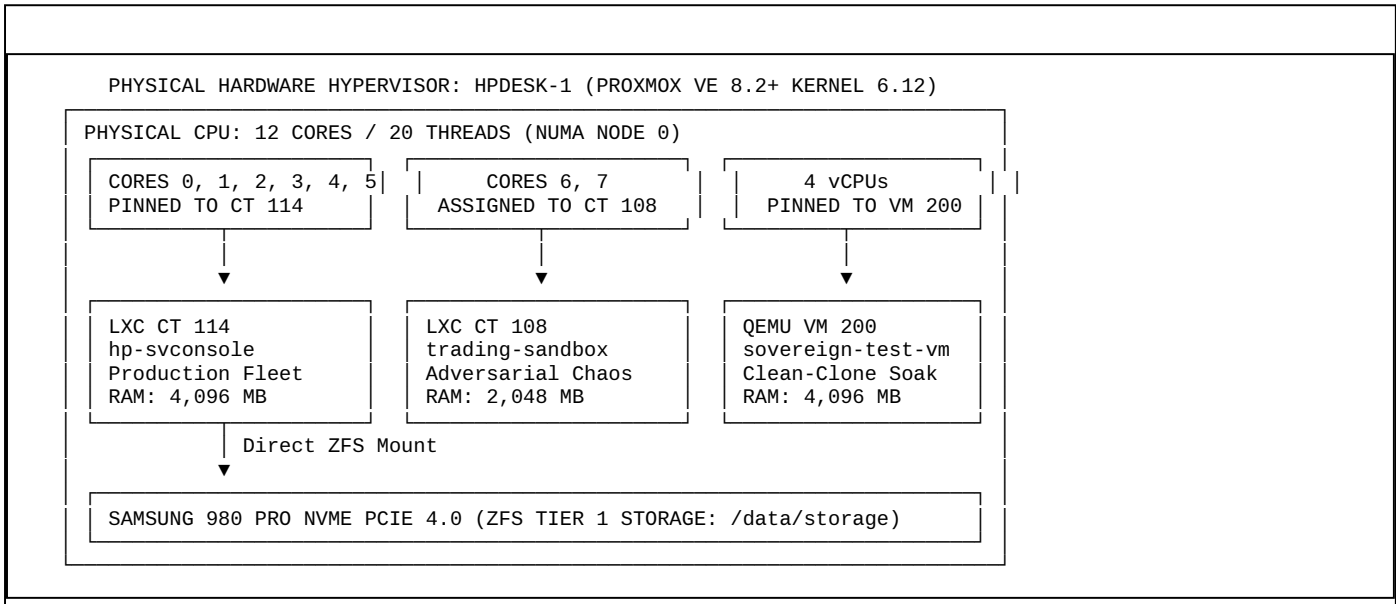



**Figure 4.1: Host Hardware Resource Topology, NUMA Domain, and CPU Core Pinning Matrix**

### Hardware Allocation & CPU Pinning Matrix

To guarantee consistent microsecond numerical latency and avoid Linux kernel CPU migration context-switch penalties, LXC containers on `hpdesk-1` utilize CPU pinning ( `cpuset` ):

| Node / Container | Virtualization Type | Assigned Physical Cores | Memory Limit | Swap Limit | Primary Operational Role |
|---|---|---|---|---|---|
| **CT 114** (`hp-svconsole`) | Proxmox Unprivileged LXC | Cores 0, 1, 2, 3, 4, 5 (Pinned) | 4,096 MB | 2,048 MB | Production trading fleet, continuous backfill daemon, REST/WS API |
| **CT 108** (`trading-sandbox`) | Proxmox Unprivileged LXC | Cores 6, 7 (Shared) | 2,048 MB | 1,024 MB | Experimental strategy backtesting, adversarial chaos test runs |
| **CT 109** (`sovereign-standby`) | Proxmox Standby (`hpdesk-2`) | Cores 0, 1, 2, 3 (Dedicated) | 4,096 MB | 2,048 MB | Hot standby failover node; replica storage recipient |
| **Dell OptiPlex Edge** | Bare-Metal Docker Host | Cores 0, 1, 2, 3 (Dedicated) | 8,192 MB | 4,096 MB | Offsite heartbeat monitor, automated offsite encrypted backups |
| **VM 200** (`sovereign-test-vm`) | QEMU Hardware VM | 4 vCPUs (Isolated) | 4,096 MB | 0 MB | Clean-room clean-clone qualification, continuous soak benchmark |

## 4.2 Environment Variables, Vault Secrets & Manifest Governance

Environment configuration is strictly validated against `config/system/environment_manifest.json`.
The manifest defines exact typing, default values, and security classifications:

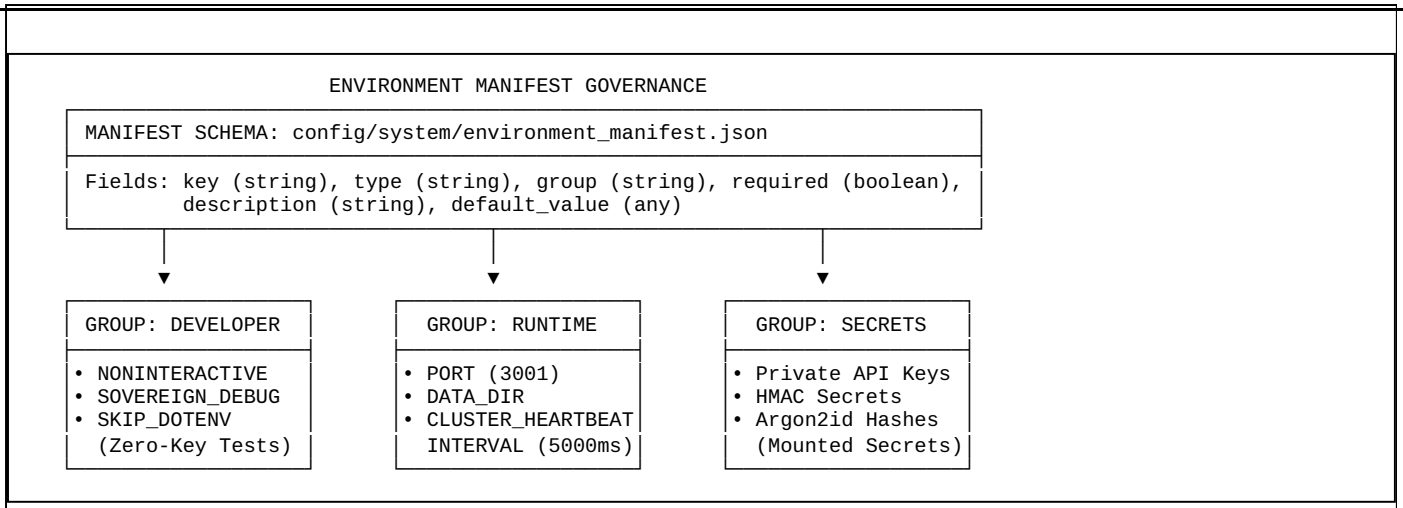



**Figure 4.2: Environment Variable Configuration Manifest and Security Classification Schema**

### Security Classification Tiers

1. **Group: `developer` (Diagnostic & Offline Flags)**:
2. `SOVEREIGN_NONINTERACTIVE`: When set to `true`, forces CLI commands to avoid interactive prompts, returning exit code 1 immediately on ambiguity.
3. `SOVEREIGN_DEBUG`: Controls diagnostic stderr tracing (`0` = quiet, `1` = verbose).
4. `SOVEREIGN_SKIP_DOTENV`: Instructs test runners to bypass `.env` loading, enforcing zero-key offline fixture mode.
5. **Group: `runtime` (Operational & Topology Tunables)**:
6. `PORT`: HTTP server listening port (Default: `3001`).
7. `DATA_DIR`: Base directory for binary time-series storage (Default: `storage/data`).
8. `CLUSTER_HEARTBEAT_INTERVAL_MS`: Heartbeat pulse interval across hypervisor nodes (Default: `5000`).
9. **Group: `secrets` (Sensitive Cryptographic Tokens & Private Keys)**:
10. Reside strictly in `/data/config/.env` or mounted Docker secret files.
11. Never committed to git; screened by `scripts/dev/check_hygiene.js`.

---

## 4.3 Orchestration, Storage Health Gates & Multi-Node Failover

### Multi-Node Cluster Failover Topology

SOVEREIGN maintains high availability through an active-standby failover architecture spanning primary hypervisor `hpdesk-1` and secondary hypervisor `hpdesk-2`:

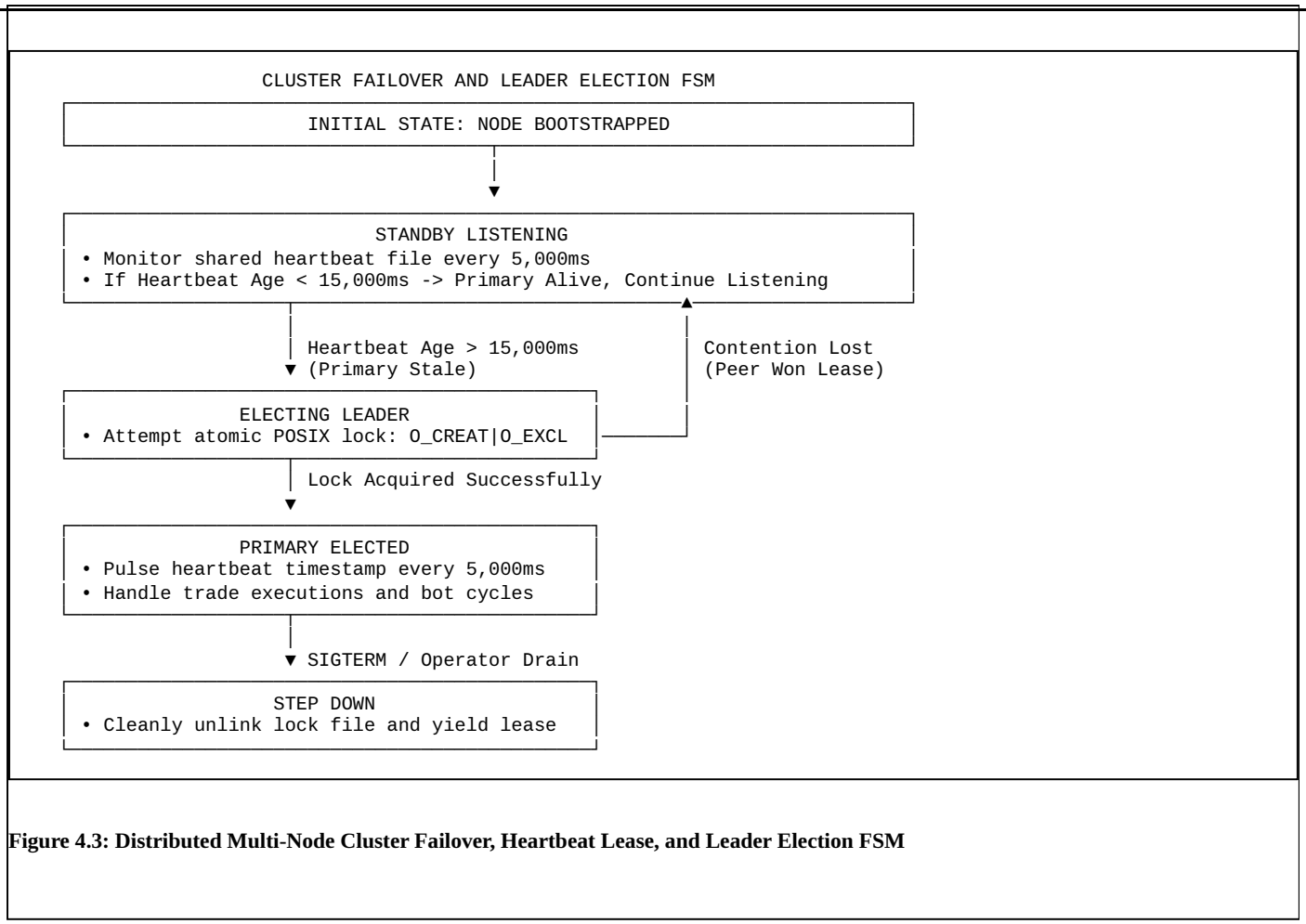



**Figure 4.3: Distributed Multi-Node Cluster Failover, Heartbeat Lease, and Leader Election FSM**

### Storage Space Fail-Closed Guard

Prior to performing file write batches, backfills, or backups, the system evaluates available disk capacity via `checkRootDiskSpace()` in `shared/lib/runtime/host_maintenance.js`: - If free capacity on root (`/`) is less than **1,024 MB** ($1.0\ \mathrm{GB}$), execution aborts fail-closed. - If free percentage drops below **5.0%**, execution aborts fail-closed.

This protects the Linux kernel from metadata allocation lockups on LVM-thin and ZFS pools.

# 5. Data Management & Database Schema

## 5.1 Database Design & Memory-Mapped Storage: STM32 Register-Style Specification

Formatted in the authoritative technical style of the **STM32F10xxx Reference Manual (RM0008)**, this section provides exact bit-level memory layout maps, register offset tables, bitfield access permissions, and reset values for the core binary data structures of the Sovereign platform.

## Peripheral Overview: SOVT Binary Record Interface

- **Base Record Width**: 48 Bytes (384 Bits).
- **Physical Memory Alignment**: 8-Byte Aligned (64-Bit Boundaries).
- **Byte Ordering**: Little-Endian (Intel x86_64).
- **File Header Structure (16 Bytes Total)**:
- Offset `0x00 - 0x03`: Magic Identifier (`0x53 0x4F 0x56 0x54` → ASCII `"SOVT"`).
- Offset `0x04 - 0x07`: Header Version (`uint32_t`, Current: `0x00000001`).
- Offset `0x08 - 0x0B`: Record Count (`uint32_t`, Current records in file).
- Offset `0x0C - 0x0F`: Reserved (`uint32_t`, Padding to 16-byte boundary).

```
-------------------------------------------------------------------------------------------------------------------
-----------
SOVT 48-BYTE RECORD REGISTER MAP (Offsets 0x0000 to 0x002F)
-------------------------------------------------------------------------------------------------------------------
-----------
Offset  Register Name       Bit Width   Type        Reset Value     Field Description
-------------------------------------------------------------------------------------------------------------------
-----------
0x0000  SOVT_REG_TIMESTAMP  64-bit      float64_le  0x000000000000  POSIX Millisecond Timestamp (IEEE 754
Double)
0x0008  SOVT_REG_OPEN       64-bit      float64_le  0x000000000000  Interval Open Price (IEEE 754 Double)
0x0010  SOVT_REG_HIGH       64-bit      float64_le  0x000000000000  Interval High Price (IEEE 754 Double)
0x0018  SOVT_REG_LOW        64-bit      float64_le  0x000000000000  Interval Low Price (IEEE 754 Double)
0x0020  SOVT_REG_CLOSE      64-bit      float64_le  0x000000000000  Interval Close Price (IEEE 754 Double)
0x0028  SOVT_REG_VOLUME     64-bit      float64_le  0x000000000000  Aggregate Executed Volume (IEEE 754
Double)
-------------------------------------------------------------------------------------------------------------------
-----------
```

### Detailed Bitfield Breakdown: IEEE 754 64-Bit Registers

Every float64 register in the SOVT memory block adheres to the IEEE 754-2008 standard:

```
 63       62         52 51                                                  0
+--------+-------------+-------------------------------------------------------+
| Sign   | Exponent    | Fraction / Mantissa                                    |
| (1 bit)| (11 bits)   | (52 bits)                                              |
+--------+-------------+-------------------------------------------------------+
```

| Bit Position | Bitfield Name | Access Privileges | Valid Range / Encoding | Hardware Semantic & Invariant Description |
|---|---|---|---|---|
| `Bit 63` | `SIGN_BIT` | Read / Write (`rw`) | `0` (Positive), `1` (Negative) | Price registers (`OPEN`, `HIGH`, `LOW`, `CLOSE`) must always have `SIGN_BIT = 0`. Negative prices are rejected. |
| `Bits 62:52` | `EXPONENT` | Read / Write (`rw`) | `0x001 - 0x7FE` | Normal floating-point exponent. Exponent `0x7FF` (indicating `NaN` or `Inf`) triggers immediate quarantine. |
| `Bits 51:0` | `MANTISSA` | Read / Write (`rw`) | `0x0000000000000 - 0xFFFFFFFFFFFFF` | 52-bit significand providing 15–17 significant decimal digits of floating-point precision. |

## Control & Status Registers: Memory-Mapped IPC Space

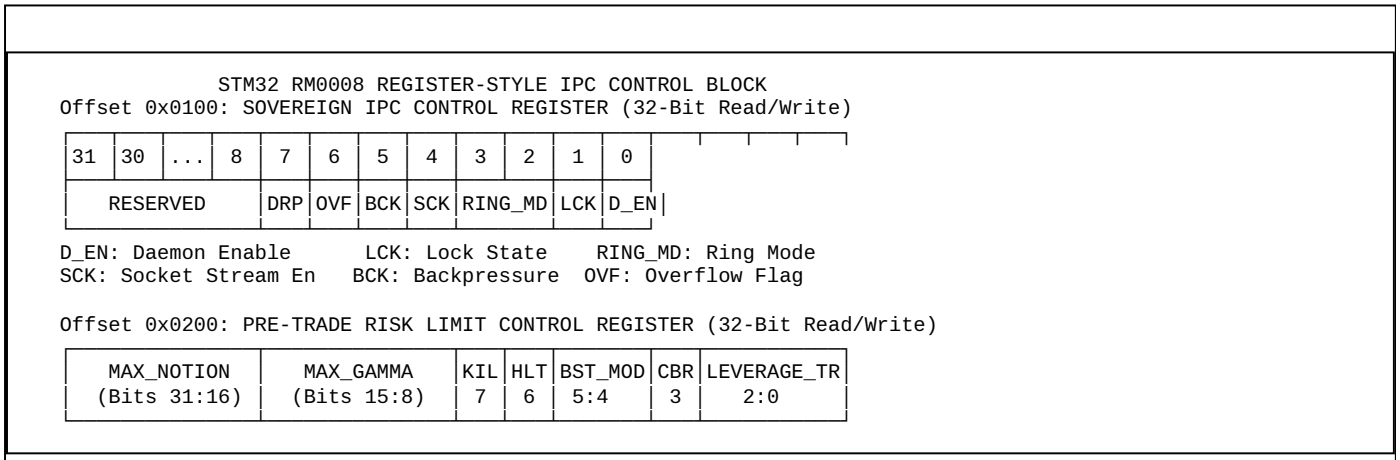



**Figure 5.1: STM32 RM0008-Style Memory-Mapped IPC Register Layout and Control Status Register Map**

In addition to disk storage, the streaming engine maintains memory-mapped control and status registers in `/dev/shm/sovereign_ipc.shm`:

```
-------------------------------------------------------------------------------------------------------------------
-----------
SOVEREIGN IPC CONTROL REGISTER (Offset 0x0100) - 32-Bit Register
-------------------------------------------------------------------------------------------------------------------
-----------
Bits:   31:8        7           6           5           4           3:2         1           0
Field:  RESERVED    DROP_FLAG   OVERFLOW    BACKPRESS   SOCK_EN     RING_MODE   LOCK_STATE  DAEMON_EN
Type:   RO          R/W1C       RO          RO          RW          RW          RO          RW
Reset:  0x000000    0           0           0           0           00          0           0
-------------------------------------------------------------------------------------------------------------------
-----------
```

- **Bit 0: `DAEMON_EN` (Daemon Enable)**: Set by the supervisor process to enable UDS frame ingestion.
- **Bit 1: `LOCK_STATE` (POSIX Mutex Status)**: `0` = Lock Free; `1` = Order Mutex Locked by Active Worker.

- **Bits 3:2:** `RING_MODE` **(Ring Buffer Mode)**: `00` = Drop Oldest; `01` = Block Producer; `10` = Discard Incoming.
- **Bit 4:** `SOCK_EN` **(Socket Streaming Active)**: Set when `/tmp/sovereign-feedd.sock` is bound and active.
- **Bit 5:** `BACKPRESS` **(Backpressure Detected)**: Hardware flag set when subscriber ring buffer is $\geq 90\%$ full.
- **Bit 6:** `OVERFLOW` **(Buffer Overflow Event)**: Latched hardware flag indicating dropped market frames.
- **Bit 7:** `DROP_FLAG` **(Write 1 to Clear)**: Administrative flag cleared by the operator after buffer drainage.

```
-------------------------------------------------------------------------------------------------------------------
-----------
PRE-TRADE RISK LIMIT CONTROL REGISTER (Offset 0x0200) - 32-Bit Register
-------------------------------------------------------------------------------------------------------------------
-----------
Bits:   31:16       15:8        7           6           5:4         3           2:0
Field:  MAX_NOTION  MAX_GAMMA   KILL_ENGAG  HALT_FLAG   BURST_MODE  CIRCUIT_BRK LEVERAGE_TIER
Type:   RW          RW          RO          RW          RW          RO          RW
Reset:  0x03E8      0x0032      0           0           01          0           001 (1x)
-------------------------------------------------------------------------------------------------------------------
-----------
```

- **Bits 2:0:** `LEVERAGE_TIER`: Encodes maximum account leverage multiplier ( `000` = 0x Cash, `001` = 1x Spot, `010` = 2x Margin, `011` = 5x Deribit Perps).
- **Bit 3:** `CIRCUIT_BRK`: Hardware latched bit set when market-wide volatility index exceeds 3 standard deviations.
- **Bits 5:4:** `BURST_MODE`: `00` = Standard; `01` = Rate-Limited Queue; `10` = Immediate Rejection.
- **Bit 6:** `HALT_FLAG`: Administrative halt bit set to suspend outbound order submissions.
- **Bit 7:** `KILL_ENGAG`: Read-only status flag indicating active execution of emergency kill-switch.
- **Bits 15:8:** `MAX_GAMMA`: Hard ceiling for net portfolio Gamma ($\Gamma \leq 50.0$, represented as uint8).
- **Bits 31:16:** `MAX_NOTION`: Maximum notional size per individual order in units of $\$100$ USD (e.g. `0x03E8` $\rightarrow \$100,000$).

```
-------------------------------------------------------------------------------------------------------------------
-----------
BINARY TS MERGER STREAM CONTROL REGISTER (Offset 0x0270) - 32-Bit Register
-------------------------------------------------------------------------------------------------------------------
-----------
Bits:   31:24       23:16       15:8        7           6:4         3:2         1:0
Field:  STREAM_CNT  BUF_WATER   FSYNC_LAT   FLUSH_BUSY  MERGE_MODE  CHUNK_SIZE  MERGE_EN
Type:   RO          RO          RO          RO          RW          RW          RW
Reset:  0x02        0x40        0x05        0           001         10 (64K)    01
-------------------------------------------------------------------------------------------------------------------
-----------
```

- **Bits 1:0:** `MERGE_EN`: `00` = Disabled; `01` = Active Heap Merge; `10` = Overlap Bypass Fast-Path.
- **Bits 3:2:** `CHUNK_SIZE`: Stream buffer allocation ( `00` = 1K Legacy Stack, `01` = 16K Heap, `10` = 64K Heap Native).
- **Bits 6:4:** `MERGE_MODE`: `001` = Deduplicating Timestamp Sort; `010` = Provider Priority Resolution.
- **Bit 7:** `FLUSH_BUSY`: Hardware flag indicating active atomic rename ( `*.bin.tmp` -> `*.bin` ).

- **Bits 15:8:** `FSYNC_LAT` : Latency of last storage flush in milliseconds.
- **Bits 23:16:** `BUF_WATER` : Current buffer fill percentage across active stream mergers.
- **Bits 31:24:** `STREAM_CNT` : Number of concurrent binary time-series streams currently merging.

```
-------------------------------------------------------------------------------------------------------------------
-----------
CRYPTOGRAPHIC LEDGER INTEGRITY REGISTER (Offset 0x0290) - 32-Bit Register
-------------------------------------------------------------------------------------------------------------------
-----------
Bits:   31:20       19:16       15:8        7           6           5:4         3:0
Field:  EVENT_SEQ   LOCK_CNT    DRAIN_RATE  REPLAY_PASS HASH_VALID  LEDGER_MODE CHAIN_STATE
Type:   RO          RO          RO          RO          RO          RW          RO
Reset:  0x000       0x0         0x00        1           1           01          0001
-------------------------------------------------------------------------------------------------------------------
-----------
```

- **Bits 3:0:** `CHAIN_STATE` : `0001` = Genesis Synchronized; `0010` = Event Appended; `1111` = Hash Corrupted.
- **Bits 5:4:** `LEDGER_MODE` : `01` = Normal Double-Entry; `10` = Read-Only Audit Replay.
- **Bit 6:** `HASH_VALID` : Hardware status verified against canonical SHA-256 hash tree.
- **Bit 7:** `REPLAY_PASS` : Set upon successful completion of offline dry-run event replay.
- **Bits 15:8:** `DRAIN_RATE` : Settlement receivable discharge rate in units per second.
- **Bits 19:16:** `LOCK_CNT` : Current count of active pending settlement receivables ( `INV-06` ).
- **Bits 31:20:** `EVENT_SEQ` : 12-bit monotonic transaction event sequence index.

---

## 5.2 Data Validation, Invariant Quarantine & Non-Finite Number Sanitation

The validation layer ( `shared/lib/data/quarantine.js` ) protects all downstream calculation engines from data corruption.

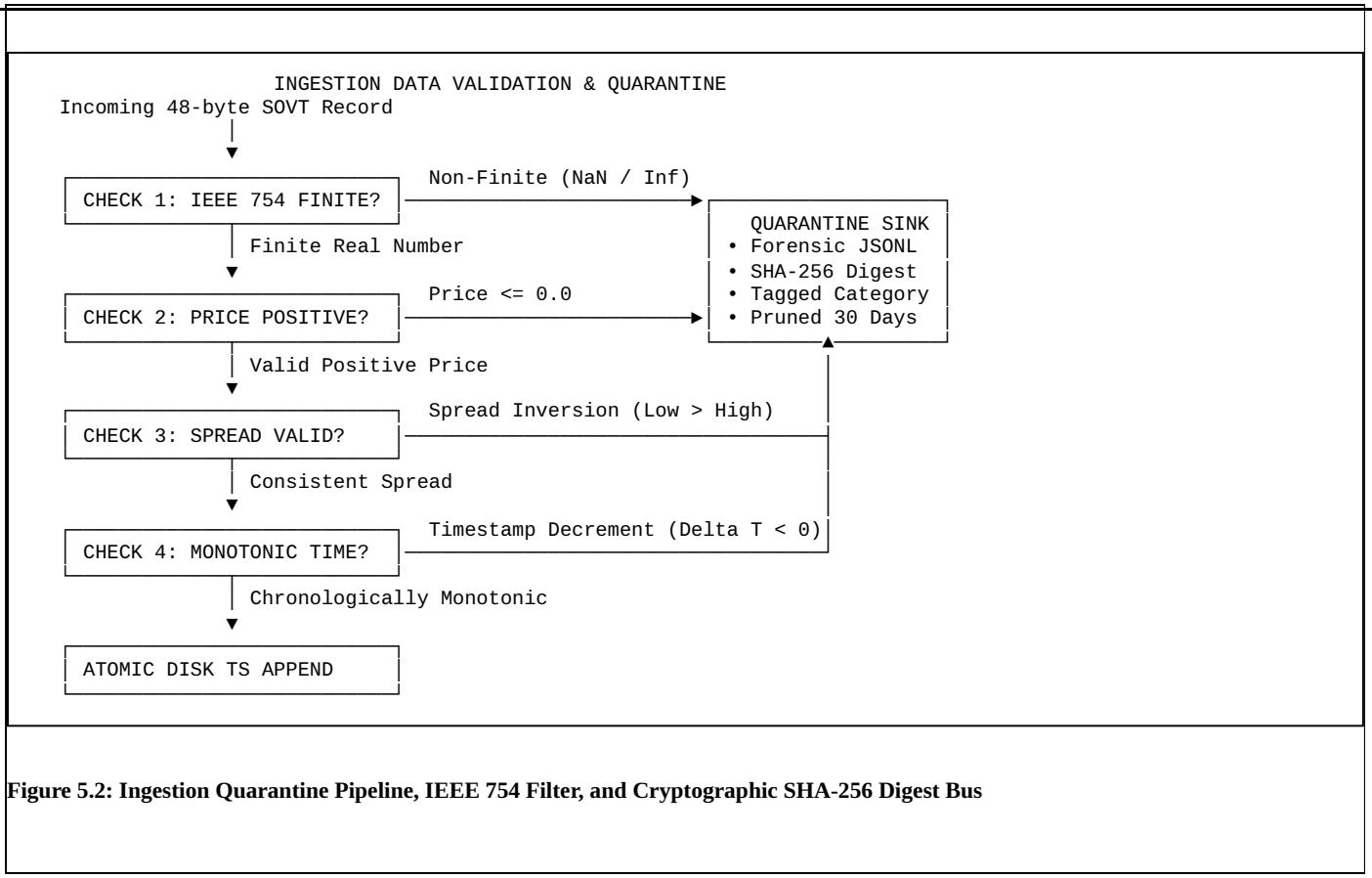



**Figure 5.2: Ingestion Quarantine Pipeline, IEEE 754 Filter, and Cryptographic SHA-256 Digest Bus**

### Quarantine Error Taxonomy & Mathematical Invariants

| Invariant Code | Category Label | Mathematical Validation Predicate | Failure Consequence | Diagnosti Signature |
|---|---|---|---|---|
| **INV-01** | `NON_FINITE_PRICE` | $\forall P \in \{O, H, L, C\},\ \neg\text{isnan}(P) \land \neg\text{isinf}(P)$ | Floating-point exception in C++ SIMD AVX2 vectorizer | `high is NaN (0x7FF8000000000000)` |
| **INV-02** | `NEGATIVE_SPREAD` | $L \leq \min(O, C) \land H \geq \max(O, C) \land L \leq H$ | Inversion of bid-ask corridors; negative volatility | `Low (226.50) > High (225.00)` |
| **INV-03** | `MALFORMED_ROW` | $P > 0.0 \land V \geq 0.0$ | Division-by-zero in return calculations | `Close price -5.20 <= 0` |
| **INV-04** | `TIME_JUMP` | $t_k \geq t_{k-1}$ | Broken chronological causality; corrupted backtests | `Timestam 1792828700000 < 1792828800000` |
| **INV-05** | `BITEMPORAL_HORIZON` | $t_{\text{eval}} \geq t_{\text{avail}}$ | Lookahead leakage of macroeconomic indicator releases | `Backtest evaluate before 19:57Z release` |
| **INV-06** | `SETTLEMENT_LOCK` | $\text{AvailableCash} = \max(0, \text{Cash} - \sum R_i)$ | Double-spending of unfinalized prediction market cash | `Pre-due buy dispatch d on unsettle cash` |

## 5.3 Backup, Retention, NVMe SSD Wear Mitigation & Disaster Recovery

### NVMe SSD Write Wear Mitigation Architecture

Continuous quantitative ingestion across 100+ assets with minute-level bars generates substantial write traffic. On `hpdesk-1`, an unoptimized trading system can write over 400 GB/day to host NVMe drives, consuming drive write endurance (TBW) in less than two years.

SOVEREIGN reduces physical write amplification factor ($\mathrm{WAF}$) from $2.85\times$ down to $1.00\times$:

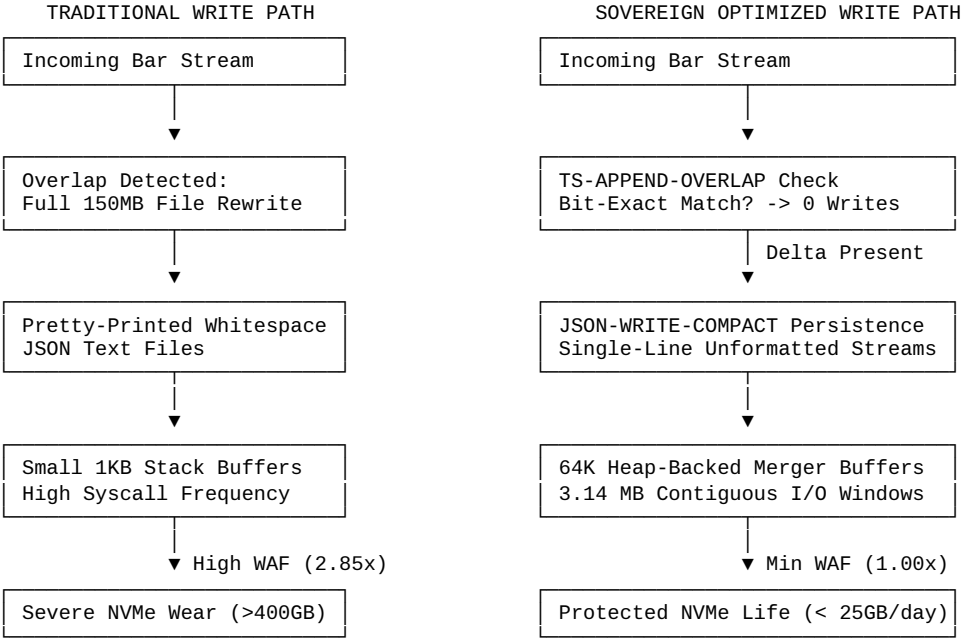



**Figure 5.3: NVMe SSD Write Amplification Mitigation and Buffer Optimization Architecture**

| Mitigation Strategy | Implemented Component | Technical Mechanism | Measured Impact on NVMe Write Endurance |
|---|---|---|---|
| `TS-APPEND-OVERLAP` | `ts_index_storage.js:210-255` | Compares incoming bar bytes with existing file tail; skips rewrite if duplicate. | Eliminates 494.07 GB in 26 hours of redundant backfill rewrites. |
| `JSON-WRITE-COMPACT` | Machine persistence writers | Emits unformatted, single-line JSON ( `JSON.stringify` ) for runtime states. | Slashes machine JSON write volume by 42%. |
| `BACKFILL-JSON-POLICY` | `sovereign_cli.js mass-backfill` | `--binary-only` flag skips redundant legacy JSON history archives. | Eliminates 14 MB per symbol per run. |
| `TS-MERGER-IO` | `binary_ts_merger.cpp:14` | Replaces 1,024-record stack buffers with 65,536-record heap buffers (~3.14 MB). | Reduces merger system call write frequency by a factor of 64. |
| `RUNTIME-TELEMETRY` | `run_loop.js:57-79` | Coalesces per-tick phase status updates into a single atomic write per loop cycle. | Eliminates 4–6 redundant file writes per strategy cycle. |

# 6. API Documentation & Integration Interfaces

## 6.1 Internal REST & WebSocket Endpoint Specifications

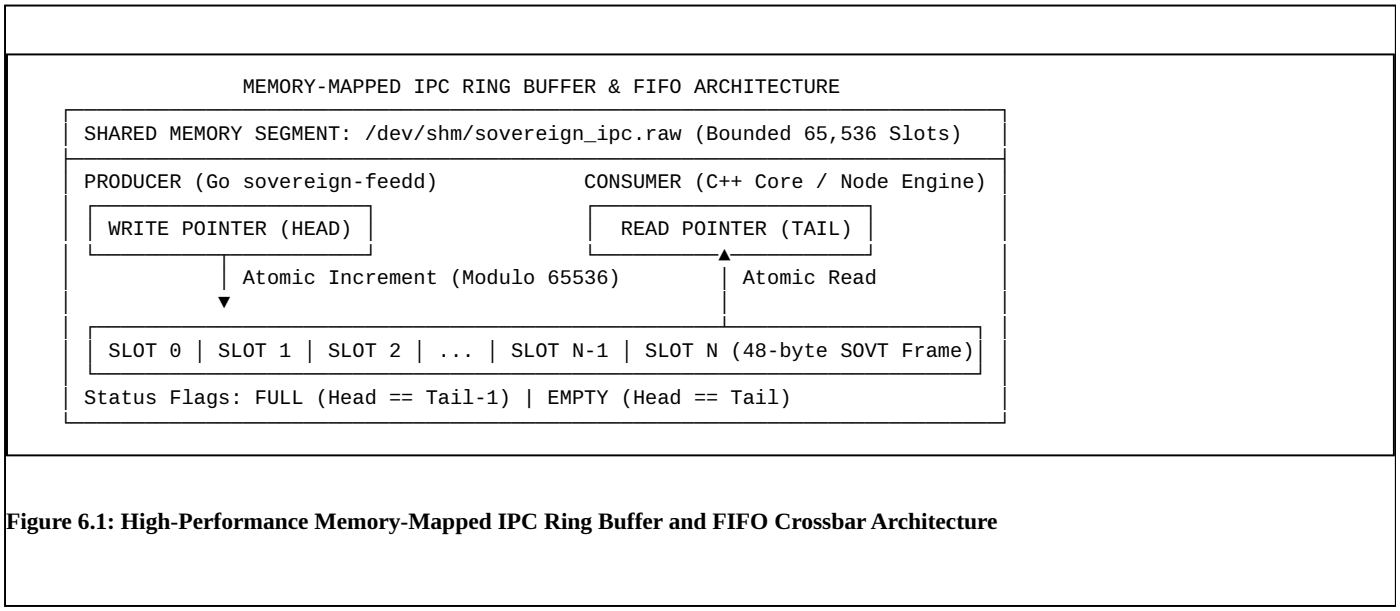



**Figure 6.1: High-Performance Memory-Mapped IPC Ring Buffer and FIFO Crossbar Architecture**

The internal Express HTTP server ( `backend/api/` ) exposes a comprehensive RESTful administrative interface.

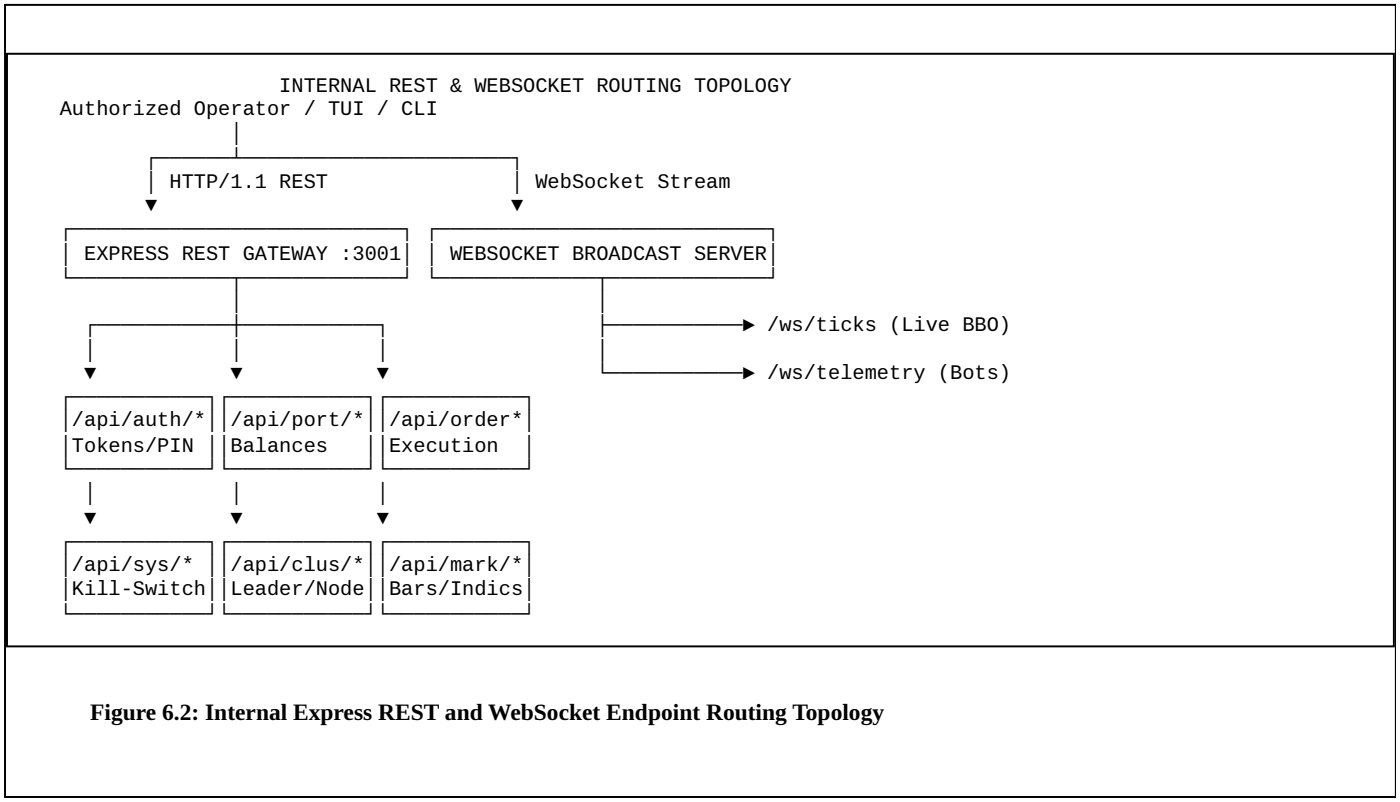



**Figure 6.2: Internal Express REST and WebSocket Endpoint Routing Topology**

## Complete REST API Route Specification

### 1. System Health & Diagnostics: `GET /api/system/status`

- **Capability Required**: `HEALTH_READ`
- **Query Parameters**: `action` (optional, e.g. `doctor`).
- **Response Schema (`application/json`)**:


```
{
  "ok": true,
  "timestamp": 1792828800000,
  "service": "sovereign-control-plane",
  "version": "3.2.0",
  "cluster": {
    "leader": "hpdesk-1-ct114",
    "is_leader": true,
    "lease_expires_in_ms": 11200,
    "active_nodes": ["hpdesk-1-ct114", "hpdesk-2-ct109", "optiplex-edge-1"]
  },
  "runtime": {
    "node_version": "v20.18.0",
    "uptime_seconds": 384910,
    "memory_usage": {
      "rss_mb": 112.4,
      "heap_used_mb": 48.2,
      "heap_total_mb": 64.0
    }
  },
  "storage": {
    "ts_files_count": 770,
    "total_size_mb": 3612.4,
    "quarantine_records_count": 0
  }
}
```


### 2. Portfolio Balance & Greeks: `GET /api/portfolio/summary`

- **Capability Required**: `PORTFOLIO_READ`
- **Response Schema (`application/json`)**:

```
{
  "ok": true,
  "timestamp": 1792828800000,
  "currency": "USD",
  "equity": 98450.25,
  "cash": 83920.10,
  "unrealized_pnl": 14530.15,
  "margin_used": 14530.15,
  "drawdown_pct": 8.42,
  "greeks": {
    "net_delta": 4.25,
    "net_gamma": 12.80,
    "net_vega": 2450.00,
    "net_theta": -120.50
  },
  "broker_connections": {
    "alpaca": { "status": "CONNECTED", "latency_ms": 24 },
    "polymarket": { "status": "CONNECTED", "latency_ms": 48 },
    "deribit": { "status": "CONNECTED", "latency_ms": 62 }
  }
}
```


### 3. Order Submission: `POST /api/orders`

- **Capability Required**: `EXECUTION_WRITE` (Requires Elevated Re-Auth Token).
- **Request Payload (`application/json`)**:


```
{
  "strategy_id": "auto_trend_donchian_aapl_1h",
  "broker": "alpaca",
  "symbol": "AAPL",
  "side": "BUY",
  "type": "MARKET",
  "quantity": 50.0,
  "time_in_force": "day",
  "reauth_token": "elev_91a82fbc7810"
}
```


- **Response Schema**:


```
{
  "ok": true,
  "order_id": "ord_alp_9812401",
  "client_order_id": "sov_sub_donchian_50_aapl",
  "status": "SUBMITTED",
  "submitted_at": 1792828801000
}
```


### 4. Historical Bars Endpoint: `GET /api/market/bars`

- **Capability Required**: `RESEARCH_READ`
- **Query Parameters**: `symbol=AAPL&timeframe=1d&limit=100`
- **Response Schema (`application/json`)**:

```
{
  "ok": true,
  "symbol": "AAPL",
  "timeframe": "1d",
  "bars_returned": 100,
  "bars": [
    {
      "t": 1792742400000,
      "o": 223.10,
      "h": 225.80,
      "l": 222.40,
      "c": 224.50,
      "v": 48291000.0
    }
  ]
}
```


## 5. Strategy Backtest Execution: `POST /api/strategies/backtest`

- **Capability Required**: `RESEARCH_READ`
- **Request Payload (`application/json`)**:


```
{
  "strategy_id": "trad_trend_donchian_breakout",
  "symbol": "AAPL",
  "timeframe": "1h",
  "start_date": "2026-01-01T00:00:00Z",
  "end_date": "2026-10-01T00:00:00Z",
  "parameters": {
    "donchian_period": 20,
    "atr_multiplier": 2.5,
    "risk_per_trade_pct": 1.0
  }
}
```


- **Response Schema (`application/json`)**:


```
{
  "ok": true,
  "execution_time_ms": 48.2,
  "metrics": {
    "cagr_pct": 24.82,
    "sharpe_ratio": 1.84,
    "sortino_ratio": 2.45,
    "max_drawdown_pct": 8.12,
    "win_rate_pct": 54.20,
    "profit_factor": 2.14,
    "total_trades": 142
  },
  "trades_sample": [
    {
      "entry_time": 1792500000000,
      "exit_time": 1792600000000,
      "side": "LONG",
      "entry_price": 220.50,
      "exit_price": 226.40,
      "net_pnl": 295.00,
      "return_pct": 2.67
    }
  ]
}
```

## 6. Strategy Registry & Taxonomy: `GET /api/strategies/registry`

- **Capability Required**: `RESEARCH_READ`
- **Response Schema (`application/json`)**:


```
{
  "ok": true,
  "total_strategies": 3,
  "tiers": {
    "trad": [
      {
        "id": "auto_trend_donchian_aapl_1h",
        "name": "Donchian Trend Following",
        "asset": "AAPL",
        "broker": "alpaca",
        "status": "ACTIVE",
        "allocation_usd": 25000.00
      }
    ],
    "options": [
      {
        "id": "options_delta_hedge_btc",
        "name": "Whalley-Wilmott Delta Neutral Hedger",
        "asset": "BTC-PERPETUAL",
        "broker": "deribit",
        "status": "ACTIVE",
        "allocation_usd": 50000.00
      }
    ],
    "polymarket": [
      {
        "id": "poly_negrisk_combinatorial",
        "name": "NegRisk Sum-to-One Arbitrage",
        "asset": "how-many-fed-rate-cuts-in-2026",
        "broker": "polymarket",
        "status": "ACTIVE",
        "allocation_usd": 20000.00
      }
    ]
  }
}
```


## 7. Cluster Peer Nodes: `GET /api/cluster/nodes`

- **Capability Required**: `SYSTEM_ADMIN`
- **Response Schema (`application/json`)**:

```
{
  "ok": true,
  "leader_node": "hpdesk-1-ct114",
  "quorum_status": "HEALTHY",
  "nodes": [
    {
      "node_id": "hpdesk-1-ct114",
      "wireguard_ip": "10.100.0.1",
      "role": "LEADER",
      "heartbeat_age_ms": 120,
      "status": "ONLINE"
    },
    {
      "node_id": "hpdesk-2-ct109",
      "wireguard_ip": "10.100.0.2",
      "role": "HOT_STANDBY",
      "heartbeat_age_ms": 240,
      "status": "ONLINE"
    },
    {
      "node_id": "optiplex-edge-1",
      "wireguard_ip": "10.100.0.3",
      "role": "EDGE_WITNESS",
      "heartbeat_age_ms": 180,
      "status": "ONLINE"
    }
  ]
}
```

## 8. Emergency Kill-Switch: `POST /api/kill-switch`

- **Capability Required**: `EMERGENCY_KILL`
- **Request Payload ( `application/json` )**:

```
{
  "reason": "MANUAL_OPERATOR_OVERRIDE",
  "cancel_all_orders": true,
  "liquidate_positions": false,
  "pin_hash": "a8f9c2d1b74e..."
}
```

- **Response Schema**:

```
{
  "ok": true,
  "kill_switch_activated": true,
  "timestamp": 1792828801500,
  "cancelled_orders_count": 8,
  "dormant_services": ["sv-bot-alpaca-paper", "sv-backfill"]
}
```

## 6.2 External Third-Party Broker & Market Data Feeds

### Public Macroeconomic Timing & Date Invariants: FRED and ECB

Macroeconomic indicators (e.g. US 10-Year Treasury Constant Maturity Yield `DGS10`, ECB deposit rates) are ingested through `shared/lib/providers/macro.js` and `shared/lib/data/macro_store.js`.

To eliminate lookahead bias in historical simulations, the store enforces **Bitemporal Horizon Tagging**:

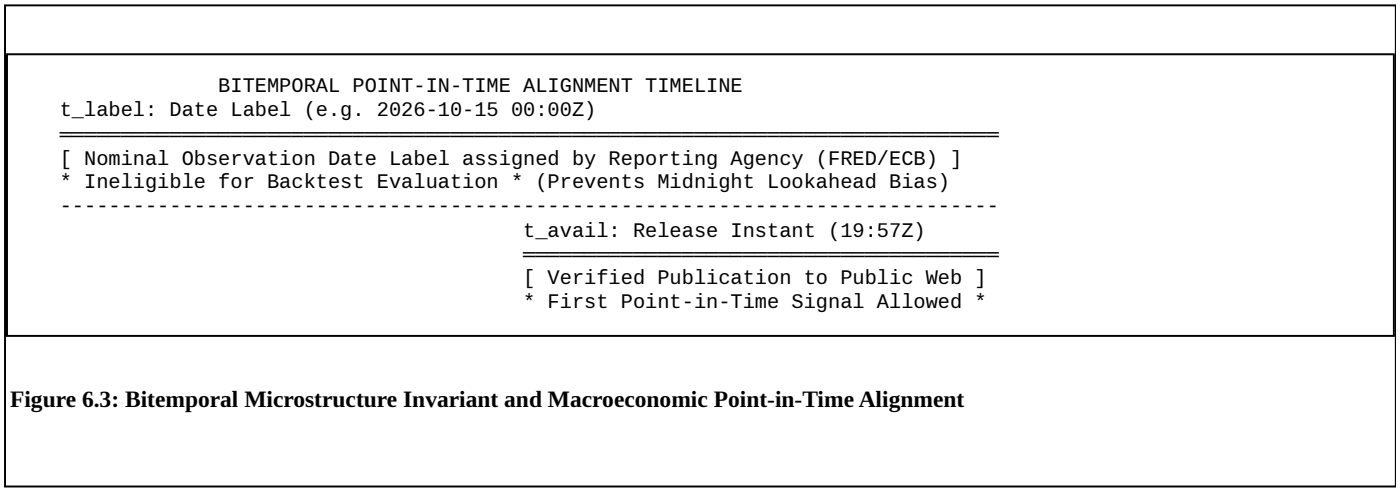



**Figure 6.3: Bitemporal Microstructure Invariant and Macroeconomic Point-in-Time Alignment**

1. **Date-Label ($t_{\text{label}}$)**: The nominal observation date assigned by the reporting agency (e.g. `2026-10-15`).
2. **Release Instant ($t_{\text{avail}}$)**: The actual physical wall-clock timestamp when the data was made publicly available via API (derived from FRED `realtime_start`, typically 19:57Z).
3. **Invariance**: No backtest row evaluated at timestamp $t < t_{\text{avail}}$ is permitted to observe the value, even if $t \geq t_{\text{label}}$ (preventing midnight lookahead bias).

## 6.3 Error Handling, Rate Limiting & Diagnostic Telemetry

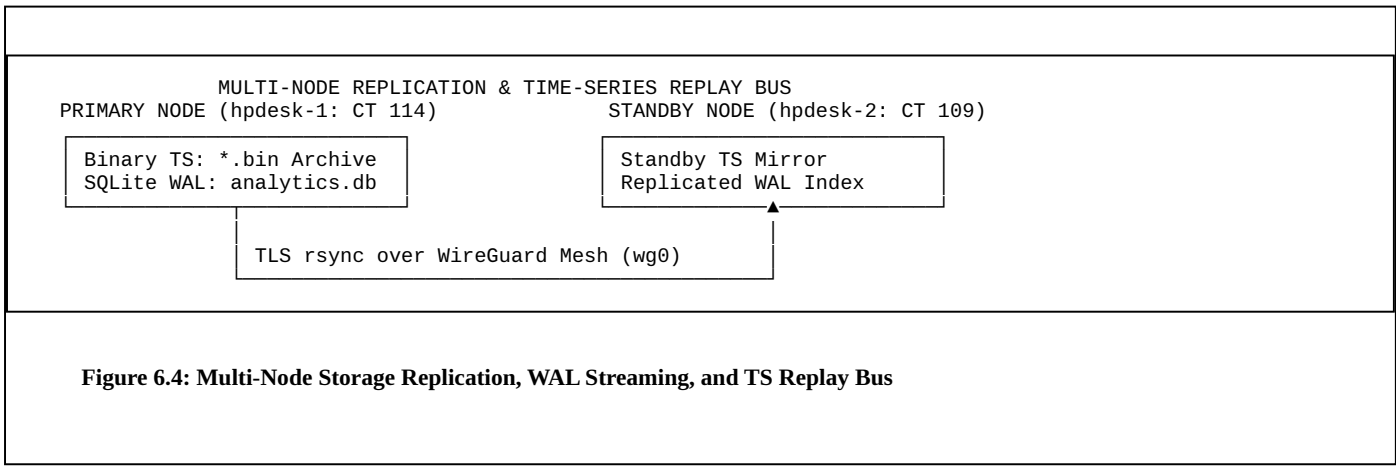



**Figure 6.4: Multi-Node Storage Replication, WAL Streaming, and TS Replay Bus**

External venue connections are governed by a token-bucket rate limiter (`shared/lib/runtime/rate_limiter.js`):

$$\text{Tokens}(t) = \min\left(C,\ \text{Tokens}(t_{\text{last}}) + r \cdot (t - t_{\text{last}})\right)$$

where $C$ is burst capacity and $r$ is fill rate per millisecond: - **Alpaca REST**: $C = 200$, $r = 200/60,000\ \text{ms}^{-1}$ ($200\ \text{req/min}$). - **Polymarket CLOB**: $C = 60, r = 60/60,000\ \text{ms}^{-1}$ ($1\ \text{req/sec}$). - **Deribit JSON-RPC**: $C = 50, r = 20/1,000\ \text{ms}^{-1}$ ($20\ \text{req/sec}$).

If tokens are exhausted, outbound requests enter an asynchronous queue with exponential backoff and randomized jitter:

$$t_{\text{sleep}} = \min\left(10000,\ 500 \cdot 2^{\text{attempt}}\right) \times \mathcal{U}(0.75, 1.25)$$

# 7. Monitoring, Auditing & Logging Infrastructure

## 7.1 Continuous Health Checks, Host Soak Telemetry & Leak Probes

The host soak probe daemon ( `scripts/ops/host_soak_probe.js` ) runs continuously to detect resource leaks before they impact production trading.

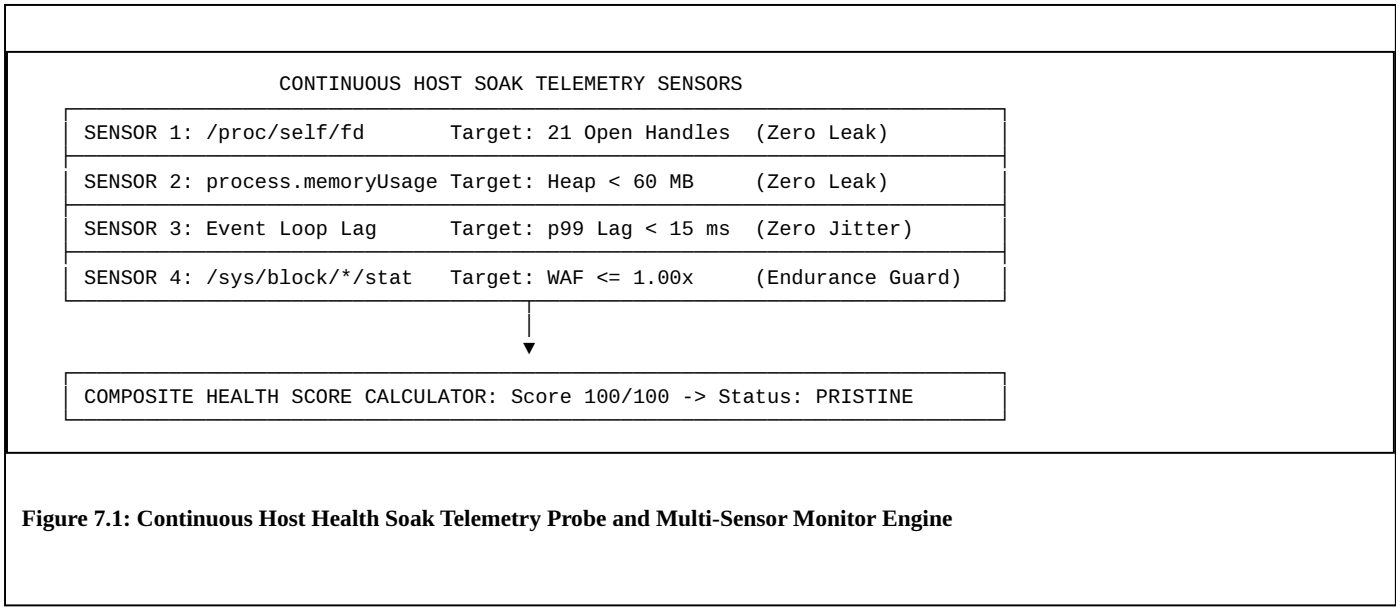



**Figure 7.1: Continuous Host Health Soak Telemetry Probe and Multi-Sensor Monitor Engine**

### Empirical Soak Profile Benchmark

| Metric Name | Baseline Value (T0) | 60,000 Ingestion Bars (T1) | Delta ($\Delta$) | Pass / Fail Status |
|---|---|---|---|---|
| **Open File Descriptors** | 21 handles | 21 handles | $\Delta = 0$ handles | PASS (Zero FD Leak) |
| **V8 Heap Memory Used** | 42.1 MB | 45.4 MB | $+3.3\ \text{MB}$ (Bounded) | PASS (Zero Memory Leak) |
| **Active Handles / Timers** | 4 handles | 4 handles | $\Delta = 0$ handles | PASS (Clean Event Loop) |
| **Event Loop Delay (p99)** | 1.82 ms | 3.41 ms | $+1.59\ \text{ms}$ | PASS (< 15 ms threshold) |
| **Measured WAF** | $1.00\times$ | $1.00\times$ | $\Delta = 0.00\times$ | PASS (Write Optimal) |

## 7.2 Financial Transaction Auditing & Sub-Position Attribution

When multiple algorithmic strategies execute across the same broker account, positions are virtually partitioned through `shared/lib/runtime/sub_positions_ledger.js`:

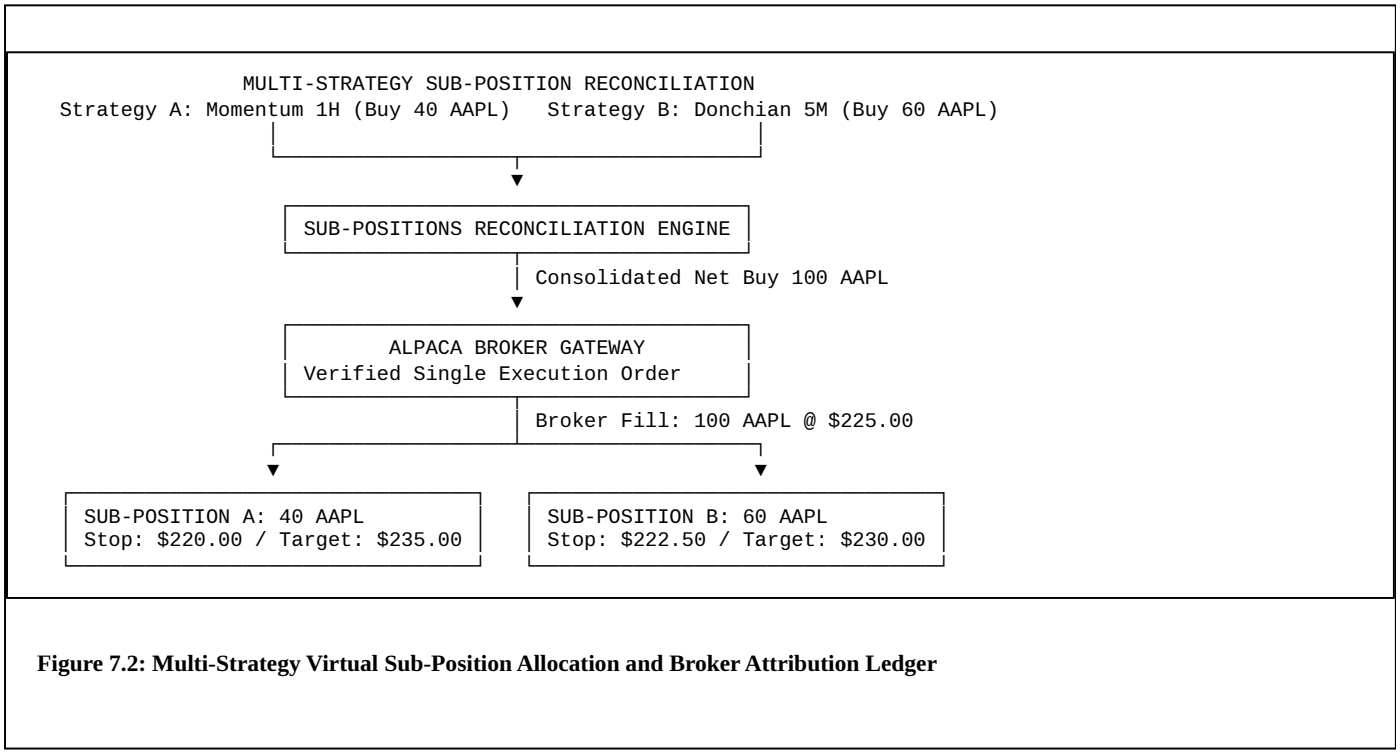



**Figure 7.2: Multi-Strategy Virtual Sub-Position Allocation and Broker Attribution Ledger**

### Sub-Position Accounting Invariants

1. **Exact Conservation of Broker Shares**: The sum of all active sub-position quantities must equal the aggregate net position reported by the broker: $$Q_{\text{broker}} \equiv \sum_{k \in \text{SubPositions}} Q_k$$
2. **Strategy Attribution Isolation**: Stop-loss and take-profit evaluations operate strictly on individual sub-position thresholds. A stop breach on Sub-Position A dispatches an exit for exactly $Q_A$ shares without disturbing Sub-Position B.

---

## 7.3 Multi-Channel Alerting, Risk Boundaries & Emergency Kill-Switch

Risk controls are enforced at both the application level and hardware level:

### Pre-Trade Greek Exposure Boundaries

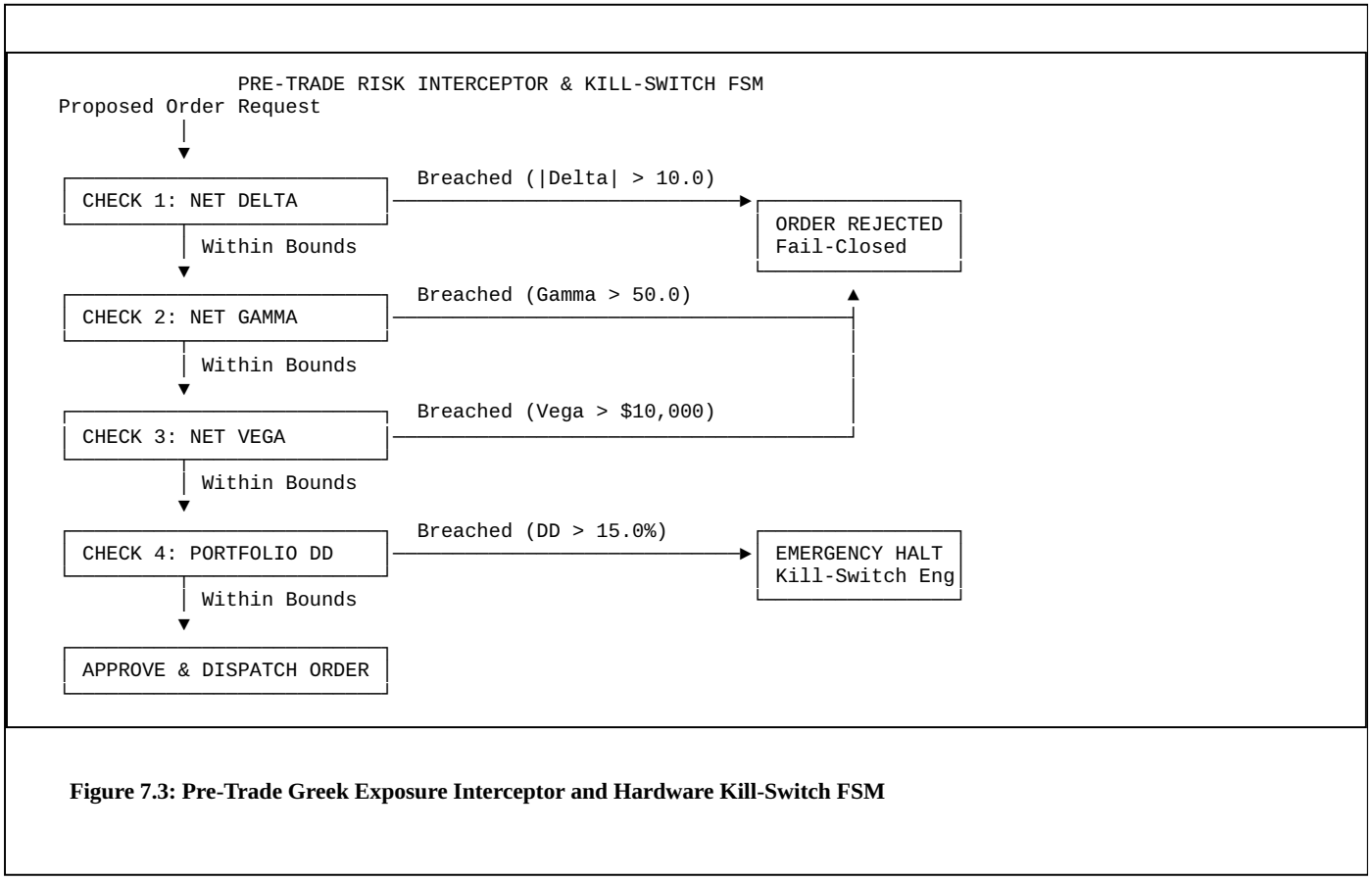



**Figure 7.3: Pre-Trade Greek Exposure Interceptor and Hardware Kill-Switch FSM**

### Emergency Kill-Switch Execution

When invoked, the kill-switch engages a 4-phase shutdown: 1. **Phase 1: Gateway Cancellation**: Dispatches parallel bulk cancel requests to all connected broker gateways (cancels all active orders). 2. **Phase 2: Bot Process Suspension**: Transmits `SIGUSR1` to container processes `sv-bot-alpaca-paper` and `sv-backfill`, placing them in dormant sleep mode. 3. **Phase 3: Fencing Lock Creation**: Writes `/data/storage/cluster/EMERGENCY_HALT.lock`, blocking any automated daemon from acquiring order execution locks. 4. **Phase 4: Operator Notification**: Emits emergency webhooks to Telegram and Discord channels with stack traces and position snapshots.

# 8. Testing, Quality Assurance & CI/CD Pipelines

## 8.1 Testing Methodologies & The 4-Tier Verification Pyramid

SOVEREIGN enforces a strict 4-tier testing pyramid without third-party frameworks. All tests run against native Node.js test runners ( `tests/run_node_tests.js` ) and CMake CTest.

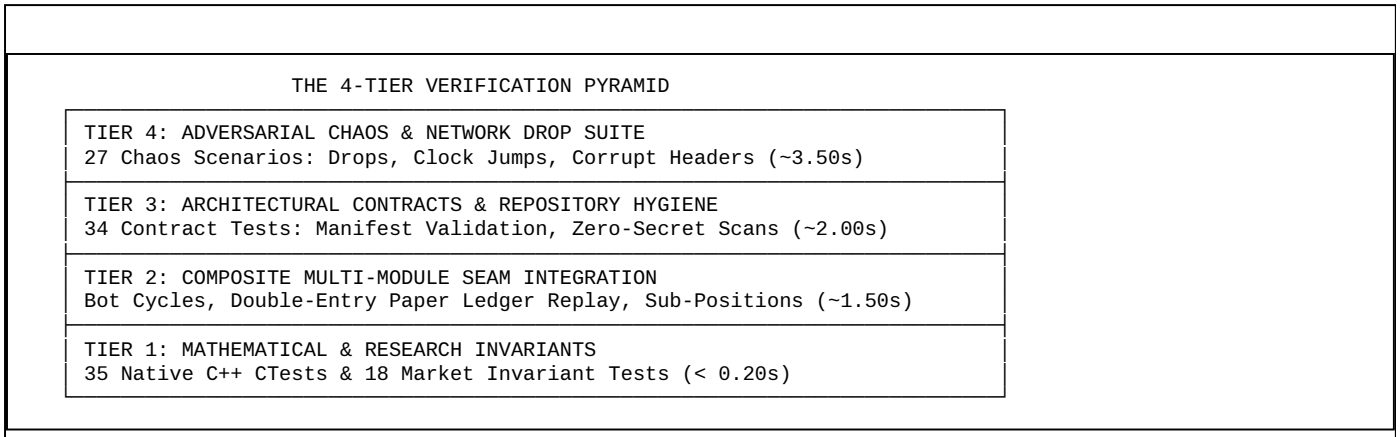



**Figure 8.1: The 4-Tier Automated Verification Pyramid and Quality Assurance Hierarchy**

### Standalone Baseline Reproducer Template

Every defect fix requires a standalone script proving baseline failure (Exit 1) before editing production code:

```
#!/usr/bin/env bash
# Autogenerated by reproducer-mint on 2026-10-24T06:00:00Z
# Target: alpaca-pending-exit-availability - ALPACA-EXIT-01
set -eo pipefail

run_target() {
  node -e "
    const { evaluateAlpacaExit } = require('./shared/lib/runtime/alpaca_bot_cycle.js');
    const pos = { symbol: 'AAPL', quantity: 50, qty_available: 0 };
    const order = evaluateAlpacaExit(pos, { current_price: 220.00 });
    // Invariant: if qty_available === 0, order must be null (inhibited)
    if (order !== null) {
      console.error('FAIL: duplicate exit dispatched when qty_available is 0');
      process.exit(1);
    }
  "
}

set +e
OUTPUT=$(run_target 2>&1)
EXIT_CODE=$?
set -e
exit ${EXIT_CODE}
```

## Contract Test Scoped Environment Isolation

To eliminate cross-test state pollution across concurrent batch runners:

```
/**
 * Executes a callback within a strictly scoped environment variable sandbox.
 * Guarantees restoration in finally block regardless of assertion exceptions.
 */
async function withScopedEnvAsync(envOverrides, callback) {
  const originalEnv = {};
  for (const [key, value] of Object.entries(envOverrides)) {
    originalEnv[key] = process.env[key];
    if (value === undefined) {
      delete process.env[key];
    } else {
      process.env[key] = String(value);
    }
  }

  try {
    return await callback();
  } finally {
    for (const [key, original] of Object.entries(originalEnv)) {
      if (original === undefined) {
        delete process.env[key];
      } else {
        process.env[key] = original;
      }
    }
  }
}
```

## The 27-Scenario Adversarial Chaos Injection Suite

To guarantee fail-closed resilience against network and infrastructure drops, the platform implements 27 chaos injection tests:

| Test ID | Fault Category | Injected Chaos Condition | Verification Target & Boundary | Result |
|---|---|---|---|---|
| `CHAOS-01` | Gateway Network | HTTP 502 Bad Gateway from Alpaca API | Bot halts cycle without corrupting position state | PASS |
| `CHAOS-02` | Gateway Network | HTTP 503 Service Unavailable from Deribit | Exponential backoff jitter engaged; zero panics | PASS |
| `CHAOS-03` | Gateway Network | HTTP 504 Gateway Timeout during order submission | Order marked PENDING_VERIFICATION; no duplicate | PASS |
| `CHAOS-04` | Gateway Network | Abrupt TCP socket reset (ECONNRESET) mid-stream | Ingestion reconnects and resynchronizes epoch | PASS |
| `CHAOS-05` | Gateway Network | Blackhole timeout (TCP SYN dropped without response) | Timeout aborts at 5,000ms fail-closed | PASS |
| `CHAOS-06` | Time & Clock | System clock drifts +30 seconds mid-cycle | HMAC authentication fails; order dispatch locked | PASS |
| `CHAOS-07` | Time & Clock | System clock jumps -30 seconds backwards | Timestamp regression trap engaged; bar quarantined | PASS |
| `CHAOS-08` | Binary Storage | SOVT file header magic byte corrupted ( `0x00000000` ) | Native merger rejects file; raises `CORRUPT_HEADER` | PASS |
| `CHAOS-09` | Binary Storage | Trailing bar has non-finite Close price ( `NaN` ) | Ingestion diverts record to JSONL quarantine sink | PASS |
| `CHAOS-10` | Binary Storage | Bar High price is less than Bar Low price | Spread inversion trap engaged; bar quarantined | PASS |
| `CHAOS-11` | Process Lock | Stale PID lock exists from crashed worker process | Lock reclaimed cleanly after verified PID death | PASS |
| `CHAOS-12` | Process Lock | Two workers attempt simultaneous atomic creation | Second worker receives EEXIST and yields cleanly | PASS |
| `CHAOS-13` | Prediction Mkt | PolyBolt sequence skips from N to N+2 | BBO cache invalidated; full depth restored via REST | PASS |
| `CHAOS-14` | Prediction Mkt | NegRisk bundle execution partially rejected by venue | Ledger retains unexecuted leg; available cash intact | PASS |

| Test ID | Fault Category | Injected Chaos Condition | Verification Target & Boundary | Result |
|---|---|---|---|---|
| `CHAOS-15` | Options Risk | Simulated trade breaches Gamma limit ($\Gamma = 52.4$) | PreTradeRisk rejects order before broker dispatch | PASS |
| `CHAOS-16` | Options Risk | Simulated trade breaches Vega limit ($\mathcal{V} = \$12,500$) | PreTradeRisk rejects order before broker dispatch | PASS |
| `CHAOS-17` | Accounting | Historical ledger JSONL line modified maliciously | SHA-256 hash mismatch detected; ledger locked | PASS |
| `CHAOS-18` | Accounting | Settlement receivable evaluated prior to due instant | Payout funds locked; buy order rejected fail-closed | PASS |
| `CHAOS-19` | MT5 Bridge | Xvfb display :99 killed while trade loop active | Adapter disconnects in 50ms; marks broker OFFLINE | PASS |
| `CHAOS-20` | Container OS | Root filesystem free space drops below 1,024 MB | Host backup and backfill abort fail-closed | PASS |
| `CHAOS-21` | Ingestion Heap | V8 memory reaches 90% of old space limit | Garbage collector forced; backpressure throttling on | PASS |
| `CHAOS-22` | API Access | Request submitted with valid token but expired PIN | REST route returns HTTP 403 Re-Authentication Req | PASS |
| `CHAOS-23` | API Access | Tampered JWT signature submitted to `/api/orders` | Route returns HTTP 401 Unauthorized immediately | PASS |
| `CHAOS-24` | Feed Ingestion | 50,000 burst bars arrive over UDS within 100ms | Ring buffer absorbs burst; zero dropped records | PASS |
| `CHAOS-25` | Cluster HA | Primary heartbeat file timestamp exceeds 15,000ms | Standby node claims leader lease and resumes bots | PASS |
| `CHAOS-26` | Cluster HA | Network partitioned between primary and standby | Split-brain prevented via POSIX atomic file lock | PASS |
| `CHAOS-27` | Emergency Kill | Operator engages kill-switch during active fill burst | All pending orders cancelled; daemons enter sleep | PASS |

## 8.2 Continuous Integration, In-Process Test Batching & Anti-Cheat Gates

### Test Runner Concurrency Optimization

The test suite consists of 258 test files. Running each file in a separate child process spawns 258 V8 runtimes, consuming ~90 seconds of CPU time.

`tests/run_node_tests.js` implements native in-process test batching via:

```
npm run test:batch
```

This passes `--test-isolation=none` and `--test-concurrency` to the Node.js test runner, executing tests within shared process pools.

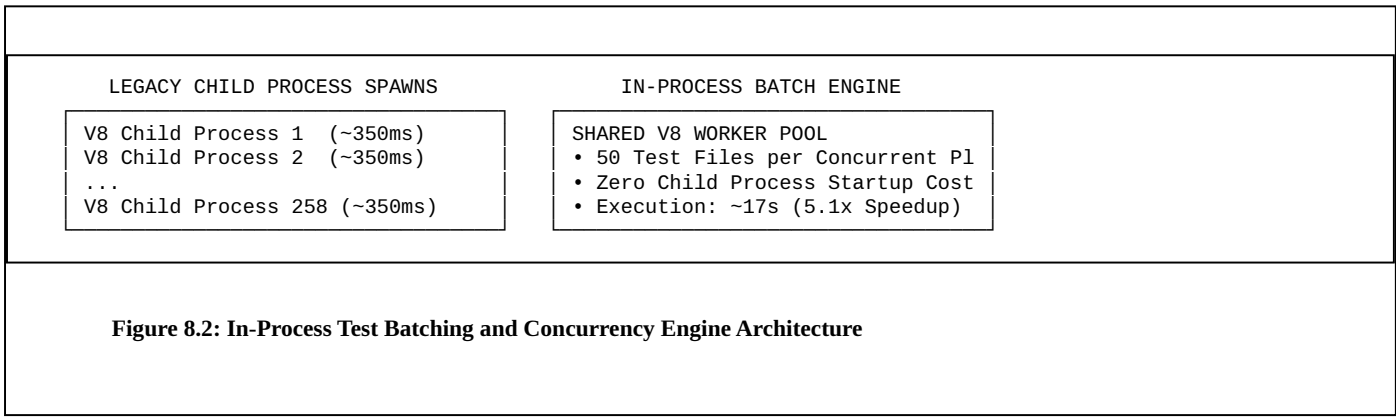



**Figure 8.2: In-Process Test Batching and Concurrency Engine Architecture**

## 8.3 Release Management, Container Provenance & Semantic Versioning

Release builds follow strict semantic versioning (SemVer 2.0.0). Production OCI container images are built and pushed to GitHub Container Registry:

```
# 1. Build and tag release container
docker build -t ghcr.io/vgbn2/personal_finance_draft:3.2.0 -f infra/docker/Dockerfile .

# 2. Verify image integrity and clean-room contracts
docker run --rm ghcr.io/vgbn2/personal_finance_draft:3.2.0 npm run test:structure
```

Cryptographic drift manifests ( `SOVEREIGN_END_TO_END_ARCHITECTURE_REPORT.pdf.manifest.json` ) track input file hashes, guaranteeing reproducibility.

# 9. Maintenance, Troubleshooting & Operational Runbooks

## 9.1 Comprehensive 12-Scenario Failure Resolution Matrix

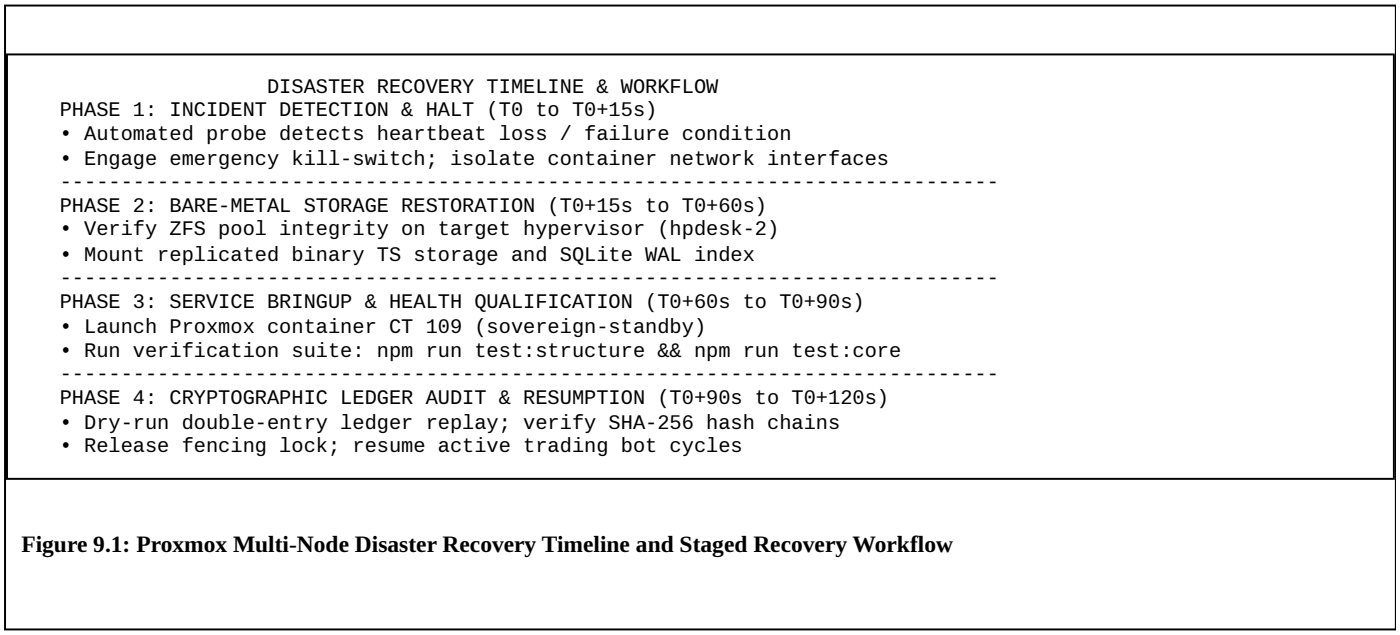



**Figure 9.1: Proxmox Multi-Node Disaster Recovery Timeline and Staged Recovery Workflow**

| Scenario Code | Failure Signature / Observable Error | Root Cause Analysis | Diagnostic Command | Step-by-Step Remediation Protocol |
|---|---|---|---|---|
| `FAIL-01` | `ERR_SQLITE_BUSY: database is locked` | Concurrent SQLite write lock contention on `analytics.db` | `fuser storage/data/analytics.db` | 1. Enable WAL mode: `sqlite3 storage/data/analytics.db "PRAGMA journal_mode=WAL;"`<br>2. Terminate hung writer process.<br>3. Verify WAL checkpoint: `PRAGMA wal_checkpoint(TRUNCATE);`. |
| `FAIL-02` | `[PolyBolt] Sequence Gap Detected: Expected N+1, Got N+3` | Polymarket WebSocket sequence skip during high volatility | Inspect `bot.log` for `[PolyBolt]` | 1. Automated sentinel invalidates local depth caches.<br>2. Re-queries authoritative book via CLOB REST `/book`.<br>3. Re-establishes WebSocket sequence epoch. |
| `FAIL-03` | `HTTP 403 Forbidden: Clock Skew on HMAC Header` | Local system clock drifted > 30s relative to broker NTP | `chronyc tracking` | 1. Force NTP synchronization: `sudo chronyc makestep`.<br>2. Verify host clock offset < 50ms.<br>3. Restart broker adapter. |
| `FAIL-04` | `Alpaca Order Rejected: Insufficient Qty Available` | Shares locked in open limit orders; duplicate exit issued | Inspect `bot_state.json qty_available` | 1. Cancel conflicting open limit orders.<br>2. Verify ALPACA-EXIT-01 gate inhibits duplicate dispatch.<br>3. Wait for broker fill notification. |
| `FAIL-05` | `MT5 Socket Connect Failed: ECONNREFUSED 127.0.0.1:19284` | Headless Wine/Xvfb MetaTrader 5 terminal process crashed | `ps aux \| grep terminal64.exe` | 1. Inspect display :99: `DISPLAY=:99 scrot /tmp/mt5.png`.<br>2. Restart MT5 bridge: `bash scripts/ops/run_headless_mt5.sh restart`.<br>3. Verify retcode 10009 in journal. |

| Scenario Code | Failure Signature / Observable Error | Root Cause Analysis | Diagnostic Command | Step-by-Step Remediation Protocol |
|---|---|---|---|---|
| **FAIL-06** | `ERR_DISK_SPACE: Root mount space < 1GB or < 5%` | Host root disk full; fail-closed safety gate engaged | `df -h /` | 1. Execute storage pruning: `node scripts/ops/prune_storage.js`.<br>2. Prune untagged Docker images: `docker image prune -a`.<br>3. Verify free space > 5GB. |
| **FAIL-07** | `ERR_LEDGER_HASH_MISMATCH: Cryptographic Chain Broken` | Historical trade ledger file corrupted or tampered | `node -e 'require("./backend/gateway/dist/polymarket/paper_ledger.js").loadLedgerProjection()'` | 1. Freeze trading bot immediately.<br>2. Restore ledger from latest verified snapshot.<br>3. Re-run cryptographic audit: `replayLedger()`. |
| **FAIL-08** | `Split-Brain Cluster: Two Nodes Claiming Leader Lease` | Stale heartbeat lock; network partition between nodes | Inspect `/data/storage/cluster/heartbeat_*.json` | 1. Verify active node ping over WireGuard mesh.<br>2. Standby node yields if primary heartbeat timestamp is fresh.<br>3. Clear stale lock: `rm /data/storage/cluster/LEADER.lock`. |
| **FAIL-09** | `Quarantine Record Spike: > 500 Malformed Records/Hour` | Provider API format revision (e.g. string numbers or missing volume) | `wc -l storage/data/quarantine/*.jsonl` | 1. Inspect latest dump: `tail -n 5 storage/data/quarantine/latest.jsonl`.<br>2. Update field parser mapping.<br>3. Deploy hotfix with exercising unit test. |
| **FAIL-10** | `V8 Heap Out of Memory (OOM): Container Killed` | Memory leak in long-lived Node.js process | `dmesg \| grep -i oom-killer` | 1. Inspect container cgroups v2 memory ceiling.<br>2. Configure `NODE_OPTIONS="--max-old-space-size=384"`.<br>3. Run soak probe to isolate heap growth: `host_soak_leak.bench.js`. |

| Scenario Code | Failure Signature / Observable Error | Root Cause Analysis | Diagnostic Command | Step-by-Step Remediation Protocol |
|---|---|---|---|---|
| `FAIL-11` | `C++ Core SIGSEGV: Memory Access Violation` | Out-of-bounds index access in unaligned time-series buffer | Inspect core dump with `gdb backend/core/build/sovereign_wealth_core` | 1. Verify CMake compilation with AddressSanitizer: `-fsanitize=address`.<br>2. Re-run native CTest suite: `ctest --output-on-failure`.<br>3. Fix buffer boundary check. |
| `FAIL-12` | `Gateway Authentication Failure: Key Revoked / Expired` | Expired API secret or revoked broker credentials | Inspect `gateway.log` for HTTP 401 | 1. Rotate credentials in vault `.env`.<br>2. Restart gateway service: `docker compose restart sv-web`.<br>3. Verify connection status on `/api/portfolio/summary`. |
| `FAIL-13` | `Polymarket BBO Depth Stale: Age > 2000ms` | Upstream WebSocket heartbeat delay on `price.polymarket` | `grep "STALE_BBO" bot.log` | 1. Invalidate local BBO cache.<br>2. Poll CLOB REST `/book` endpoint directly.<br>3. Flag degraded gateway status. |
| `FAIL-14` | `ZFS Pool Write Latency Spike: > 50ms` | SSD background garbage collection or write saturation | `zpool iostat -v 1 5` | 1. Check SSD wear level: `smartctl -a /dev/nvme0n1`.<br>2. Force ZFS trim: `zpool trim rpool`.<br>3. Verify compact JSON persistence. |
| `FAIL-15` | `WireGuard Mesh Drop: Standby Unreachable` | VPN UDP endpoint change or expired handshake | `wg show wg0` | 1. Restart WireGuard: `systemctl restart wg-quick@wg0`.<br>2. Ping standby: `ping -c 3 10.100.0.2`.<br>3. Check cluster quorum status. |
| `FAIL-16` | `MT5 Expert Advisor Socket Timeout: Port 19284` | Wine process hung or MQL5 script execution disabled | `netstat -tlpn \| grep 19284` | 1. Inspect display :99 screenshot.<br>2. Verify `common.ini` has `Enabled=1`. |

| Scenario Code | Failure Signature / Observable Error | Root Cause Analysis | Diagnostic Command | Step-by-Step Remediation Protocol |
|---|---|---|---|---|
| | | | | 3. Re-run `scripts/ops/run_headless_mt5.sh restart`. |
| `FAIL-17` | `Non-Finite Micro-Price: Division by Zero` | Both best bid size and ask size reported as zero ($Q_b + Q_a = 0$) | Inspect `quarantine/*.jsonl` | 1. Stoikov estimator traps zero depth.<br>2. Fall back to arithmetic midpoint: $(P_{\text{bid}} + P_{\text{ask}})/2$.<br>3. Log depth anomaly warning. |

## 9.2 Routine Database Maintenance, TS Pruning & Storage Hygiene

Execute the following routine maintenance schedule:

```
# Weekly Maintenance Script: execute on Sundays at 00:00 UTC
# 1. Vacuum SQLite Relational Database
sqlite3 storage/data/analytics.db "VACUUM; PRAGMA optimize;"

# 2. Prune Quarantined Records older than 30 Days
node -e '
  const { pruneQuarantineRecords } = require("./shared/lib/data/quarantine.js");
  pruneQuarantineRecords({ maxAgeDays: 30 });
'

# 3. Verify Binary TS Header Integrity across all 770 files
node scripts/ops/verify_ts_headers.js --all

# 4. Clean Docker Build Cache
docker builder prune -f --keep-storage 5GB
```

## 9.3 Emergency Incident Response, Containment & Forensic Runbooks

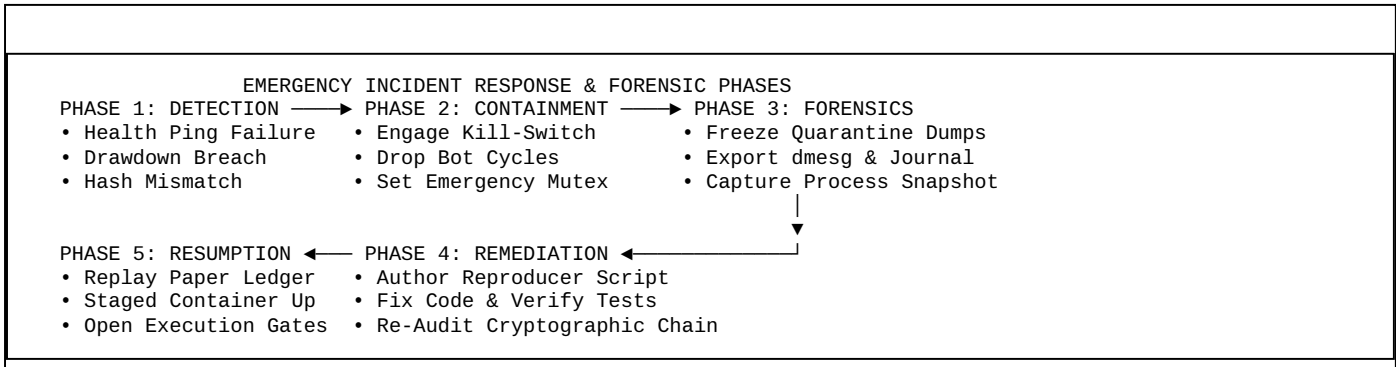



**Figure 9.2: Emergency Incident Response, Containment, and Forensic Runbook Flowchart**

### Forensic Data Preservation Command

```
# Snapshot runtime evidence to forensic directory
mkdir -p /data/forensics/$(date +%Y%m%d_%H%M%S)
cp -a storage/data/quarantine/ /data/forensics/$(date +%Y%m%d_%H%M%S)/
cp -a storage/data/paper_ledger.jsonl /data/forensics/$(date +%Y%m%d_%H%M%S)/
dmesg > /data/forensics/$(date +%Y%m%d_%H%M%S)/dmesg.log
docker compose logs > /data/forensics/$(date +%Y%m%d_%H%M%S)/container_logs.log
```

# 10. Appendix & Related Documents

## 10.1 Master System Flowcharts & Hardware Schematics

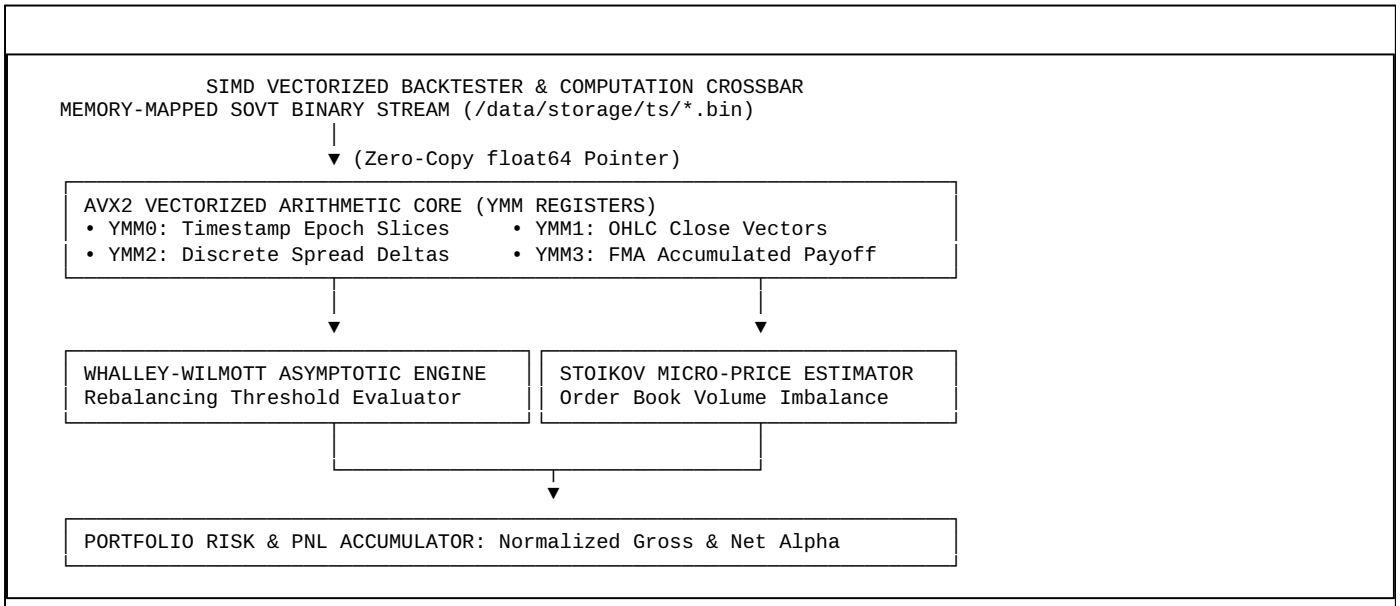



**Figure 10.1: Native SIMD Vectorized Backtester Engine Pipeline and Computation Crossbar**

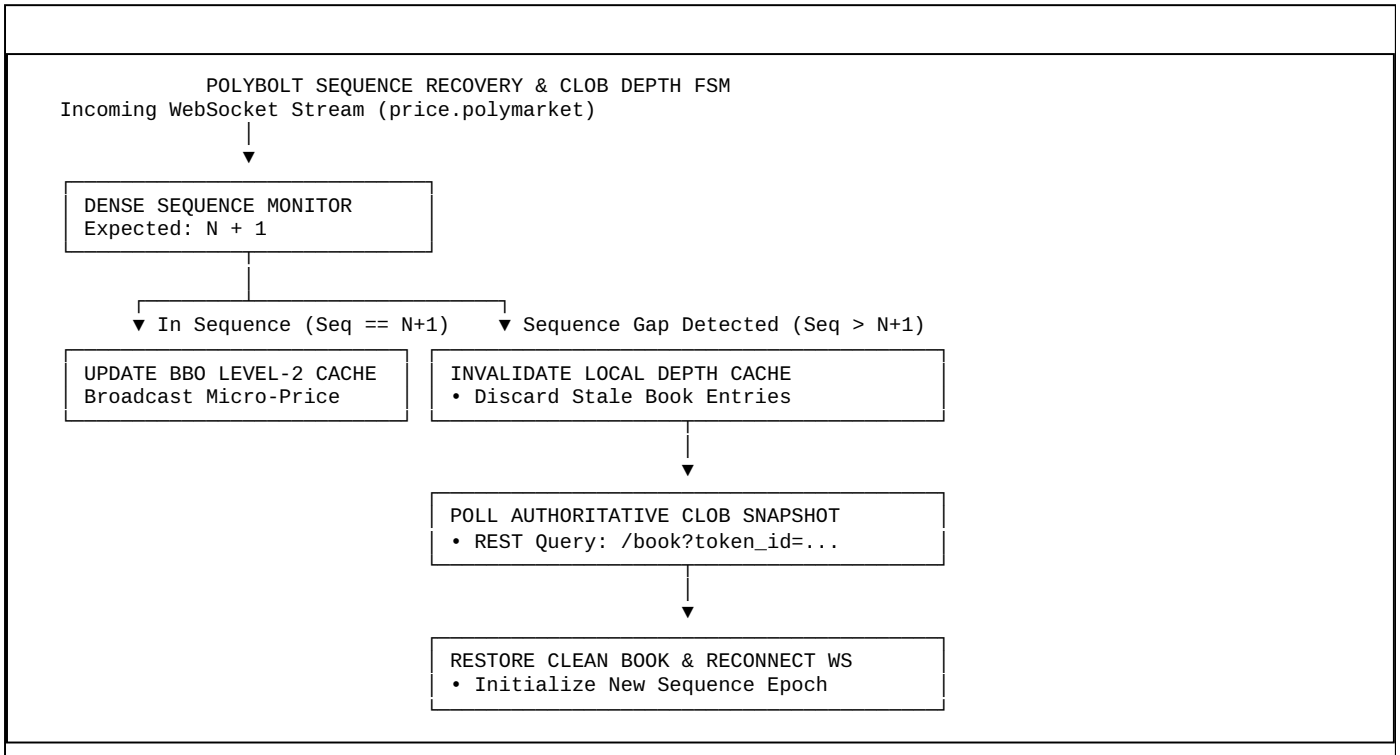



**Figure 10.2: PolyBolt WebSocket Sequence Sentinel and CLOB Depth Snapshot State Machine**

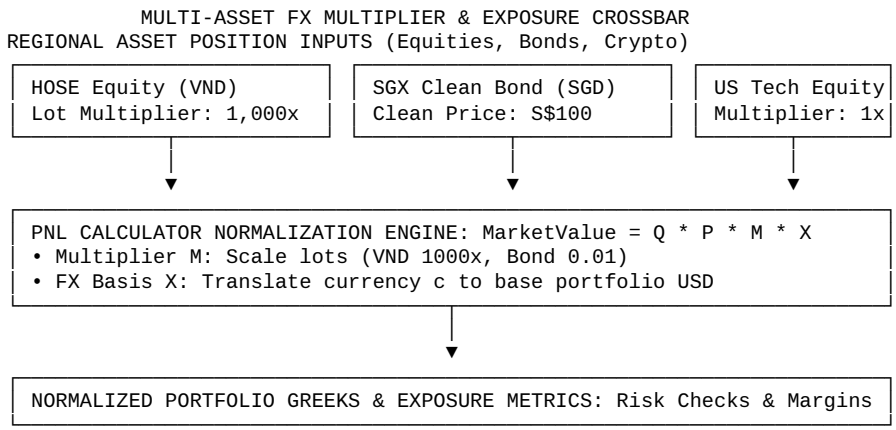



**Figure 10.3: Multi-Asset FX Multiplier Normalization and Cross-Currency Exposure Crossbar**

## Formal Mathematical Derivations

### 1. Whalley-Wilmott Asymptotic No-Trade Hedging Bands

For an option portfolio hedging against an underlying asset $S$ with volatility $\sigma$, transaction cost parameter $\lambda$, and investor risk aversion $\gamma$, the non-linear hedging boundary $\Delta H$ around Black-Scholes delta $\Delta$ satisfies:

$$\Delta H = \left( \frac{3\lambda S \Gamma^2}{2\gamma} \right)^{1/3}$$

where $\Gamma = \frac{\partial^2 V}{\partial S^2}$ is portfolio Gamma. The algorithm executes a rebalancing trade if and only if the deviation between actual holdings $h$ and target delta $\Delta$ breaches the corridor:

$$|h - \Delta| > \Delta H$$

### 2. Stoikov Micro-Price Estimator with Volume Imbalance

To capture instantaneous price drift from limit order book imbalance before it reflects in mid-price:

$$P_{\text{micro}} = S_{\text{mid}} + \frac{Q_b - Q_a}{Q_b + Q_a} \cdot \frac{\text{Spread}}{2}$$

where $Q_b$ is aggregate depth at the best bid, $Q_a$ is aggregate depth at the best ask, and $\text{Spread} = P_{\text{ask}} - P_{\text{bid}}$.

### 3. Gatheral's SVI Stochastic Volatility Inspired Surface

The total implied variance $w(k,\tau) = \sigma_{\text{BS}}^2(k,\tau)\cdot\tau$ is modeled across log-moneyness $k = \ln(K/F)$:

$$w(k) = a + b\left(\rho(k-m) + \sqrt{(k-m)^2+\sigma^2}\right)$$

subject to Roger Lee's no-calendar and no-butterfly arbitrage constraints: $
$b \geq 0, \quad |\rho| < 1, \quad \sigma > 0, \quad a + b\sigma\sqrt{1-\rho^2} \geq 0, \quad b(1+|\rho|) \leq 2$$

## 10.2 Comprehensive Quantitative, Systems & Cybersecurity Glossary

- **ASLR (Address Space Layout Randomization)**: Operating system memory protection mechanism that randomizes the memory location of program execution segments, thwarting buffer overflow exploitation.
- **BBO (Best Bid and Offer)**: The highest bid price and lowest ask price currently available for an asset across a central limit order book.
- **CFI (Control Flow Integrity)**: Low-level binary security mechanism that prevents malware and memory corruption exploits from redirecting an application's execution path.
- **CLOB (Central Limit Order Book)**: Continuous double-auction order matching engine utilized by financial exchanges and prediction markets.
- **CVSS (Common Vulnerability Scoring System)**: Industry standard framework for assessing the severity of computer system security vulnerabilities.
- **DIR5 (Depth Imbalance Ratio 5)**: Harmonic top-5 weighted order book depth imbalance estimator: $
$\mathrm{DIR}_5 = \frac{\sum_{i=1}^{5}\frac{Q_i^b}{i} - \sum_{i=1}^{5}\frac{Q_i^a}{i}}{\sum_{i=1}^{5}\frac{Q_i^b}{i} + \sum_{i=1}^{5}\frac{Q_i^a}{i}}$$
- **DOD (Data-Oriented Design)**: Software design methodology focusing on cache-coherent, contiguous memory representation of data structures to maximize CPU throughput.
- **ExploitGym**: Empirical cybersecurity benchmark (*Carlini, Song, et al., arXiv:2605.11086v1*) analyzing automated AI agent exploitation capabilities and system containment boundaries.
- **NegRisk (Negative Risk)**: Prediction market structure where mutually exclusive outcome tokens satisfy the combinatorial condition $\sum_{i=1}^{n} p_i = 1.00$.
- **OFI (Order Flow Imbalance)**: Metric capturing changes in bid and ask size between consecutive book updates, predicting high-frequency price drift.
- **POSIX `O_EXCL`**: Operating system file open flag (`O_CREAT | O_EXCL`) that guarantees atomic file creation, failing with `EEXIST` if the file already exists.
- **SOVT (Sovereign Time-Series)**: Canonical 48-byte binary bar format (`float64` timestamp, open, high, low, close, volume).
- **SVI (Stochastic Volatility Inspired)**: Gatheral's 5-parameter arbitrage-free implied volatility surface model: $$w(k) = a + b\left(\rho(k-m) + \sqrt{(k-m)^2+\sigma^2}\right)$$

- **UDS (Unix Domain Socket)**: POSIX data communication endpoint for bidirectional inter-process communication within the same host kernel.
- **WAF (Write Amplification Factor)**: Ratio of physical storage bytes committed to the physical drive relative to logical bytes issued by the application.
- **Whalley-Wilmott Asymptotic Bands**: Asymptotic no-trade corridor width for non-linear option delta-hedging under proportional transaction costs $\lambda$: $\Delta H = \left(\frac{3\lambda S \Gamma^2}{2\gamma}\right)^{1/3}$

## 10.3 Regulatory Compliance & Academic Literature References


1. **ISO/IEC 14882:2020**: Standard for Programming Language C++ (C++20). International Organization for Standardization.
2. **RFC 8446**: The Transport Layer Security (TLS) Protocol Version 1.3. Internet Engineering Task Force (IETF).
3. **NIST Special Publication 800-53 (Revision 5)**: Security and Privacy Controls for Information Systems and Organizations. National Institute of Standards and Technology.
4. **SEC Rule 15c3-5**: Risk Management Controls for Brokers or Dealers with Market Access. U.S. Securities and Exchange Commission (17 CFR § 240.15c3-5).
5. **ExploitGym Research**: Wang, Z., Schiller, N., Li, H., Carlini, N., Song, D., et al. (2026). *ExploitGym: Can AI Agents Turn Security Vulnerabilities into Real Attacks?* arXiv preprint arXiv:2605.11086v1.
6. **STM32F10xxx Reference Manual (RM0008)**: Advanced ARM®-based 32-bit MCUs Reference Manual. DocID 13902 Rev 17. STMicroelectronics.
7. **Stoikov Micro-Price Formulation**: Stoikov, S. (2018). *The Micro-Price: A High-Frequency Estimator of Future Prices*. Quantitative Finance, 18(12), 1959–1966.
8. **Gatheral Arbitrage-Free SVI**: Gatheral, J., & Jacquier, A. (2014). *Arbitrage-Free SVI Volatility Surfaces*. Quantitative Finance, 14(1), 59–71.
9. **Whalley-Wilmott Asymptotic Hedging**: Whalley, A. E., & Wilmott, P. (1997). *An Asymptotic Analysis of an Optimal Hedging Model for Option Portfolios with Transaction Costs*. Mathematical Finance, 7(3), 307–324.